
Rearrangement Planning for Construction Assembly using Teams of Multirotors

Neuordnungsplanung im Bauwesen mithilfe von Teams von
Multikoptern

Theo Valentin Kern

Universitätsmasterarbeit
zur Erlangung des akademischen Grades

Master of Science
(M. Sc.)

im Studiengang
Computer Engineering

eingereicht am 1. November 2023 am
Fachgebiet Learning and Intelligent Systems der
Fakultät Elektrotechnik und Informatik
der Technischen Universität Berlin

Betreuer

Dr. Wolfgang Hönig
Dr. Andreas Orthey

Gutachter

Dr. Wolfgang Hönig
Prof. Dr. Marc Toussaint

Abstract

Autonomous construction assembly is one of the most promising research areas to minimize the required manual labor in this industry. One of the integral challenges is the efficient rearrangement of building blocks. To effectively apply autonomous construction assembly in practice, it is necessary to utilize multiple agents, allowing for an increase in construction speed through parallelization, as well as solving more complex construction tasks by joining different robots' workspaces.

While solving the problem is already hard for one agent, prior work addressed it for a heterogeneous team of mobile manipulator arms by applying an integrated Task and Motion Planning approach that decomposes the problem into subproblems and iteratively solves them. This thesis applies previous work to coordinate teams of multi-rotors and extends it by coalition formation, which involves multiple robots to lift a single object. We present four heuristics that extract subproblems and assign robots to solve them, demonstrate these on multiple case studies, and evaluate them based on total robot workload and scalability performance. We can show that the total construction time is not significantly influenced by the choice of the heuristic and is mainly affected by the scenario structure and number of construction robots. Moreover, the planning time primarily depends on the number of quadrotors in use and secondarily on the quadrotor saturation in the scenario. The results indicate that quadrotors could be a vital part of future automated construction assembly.

Zusammenfassung

Die autonome Montage von Bauwerken ist eines der vielversprechenden Forschungsgebiete, um die erforderliche manuelle Arbeit in dieser Branche zu minimieren. Architekten können die Fähigkeiten von Robotern bereits im Entwurfsprozess berücksichtigen und diese für eine optimierte Bauphase nutzen.

Eine der wesentlichen Herausforderungen ist die effiziente Neuordnung von Bausteinen. Um die autonome Baumontage in der Praxis sinnvoll anwenden zu können, müssen mehrere Agenten eingesetzt werden. Dies erhöht die Bausgeschwindigkeit durch Parallelisierung und ermöglicht durch die Zusammenlegung der Workspaces unterschiedlicher Roboter die Umsetzung komplexerer Bauaufgaben. Obwohl das Planen des Bauvorganges bereits für einen einzelnen Agenten schwer ist, betrachten frühere Arbeiten ein heterogenes Team von mobilen Roboterarmen und haben eine Lösung für deren kooperative Koordination gefunden. Sie verwenden einen integrierten Ansatz des Task and Motion Planning, um das Problem in Teilprobleme zu zerlegen. In der vorliegenden Arbeit werden frühere Ausarbeitungen zur Koordinierung von Teams aus Multirotoren angewandt und durch Coalition Formation ergänzt.

Wir stellen vier Heuristiken vor, die Teilprobleme extrahieren und den Robotern zuweisen, um sie zu lösen. Diese demonstrieren wir anhand mehrerer Beispielszenarien und werten sie bezüglich der Gesamtarbeitslast der Roboter und Skalierbarkeit aus. Die Gesamtbauzeit eines Szenarios ist in erster Linie von der Anzahl der eingesetzten Quadratoren sowie der Quadrator-Sättigung im Szenario abhängig. Wir zeigen, dass sie durch die Wahl der Heuristik nicht signifikant beeinflusst wird. Die Planungszeit ist dabei hauptsächlich von der Anzahl der Quadrotoren abhängig, wird aber auch vom Aufbau der Szenarien beeinflusst.

Acknowledgments

I want to thank the Learning and Intelligent Systems research lab for the support throughout the work on this thesis and the project from which it is derived. I want to express my deepest appreciation to Wolfgang Hönig and Andreas Orthey. They not only inspired me to write this thesis with their lecture *Motion Planning* but also accompanied me on the whole to-be-optimized trajectory of this project by willingly sharing their knowledge, feedback, and wisdom.

I thank the company I work for, R3 Solutions, for allowing me to work in their office, with special thanks to Mathias Bohge, Sebastian Kruse, and my fellow colleagues Tobias Neubert and Anouk Franke for challenging my approaches, the productive and joyful atmosphere and their mental support during rough times.

Special thanks also go to Jarek Liesen, Helena Melusine Kern, and Simon Wietheger for taking much time and effort to proofread this thesis.

Last but not least, I want to thank Thomas Bläsius and Martin Krejca for providing the template used in this thesis.

Contents

Abstract	3
Zusammenfassung	5
Acknowledgments	7
Contents	9
1 Introduction	1
1.1 Contribution	2
1.2 Methodology and Structure	2
I Background	5
2 Motion Planning Basics	7
2.1 Problem Definition	7
2.2 Search-based Planning	9
2.3 Optimization-based Planning	11
2.3.1 Covariant Hamilton Optimization for Motion Planning .	12
2.3.2 Sequential Convex Programming	12
2.4 Sampling-based Planning	13
2.4.1 Rapidly-Exploring Random Tree	14
2.4.2 Probabilistic Roadmap Method	14
2.5 Sampling-based Algorithm Variants	16
2.5.1 RRT* and PRM*	16
2.5.2 RRT-Connect	18
2.5.3 Goal Bias	19
2.5.4 Alternative Sampling Methods	19
2.5.5 LazyPRM	20
2.5.6 Optimality	21
2.6 Comparison	21

3	Task and Motion Planning	23
3.1	Discrete Task Planning	23
3.1.1	Rearrangement Planning	24
3.2	Task and Motion Planning	25
3.2.1	Multi Robot Task Planning	25
4	Controlling a Quadcopter	27
4.1	Quadcopter Dynamics	27
4.2	Controllers	29
4.2.1	Basic Building Blocks	30
4.2.2	Control of the Bitcraze Crazyflie	30
II	Fundamental Building Blocks	33
5	Space-Time RRT*	35
5.1	Features and Challenges	35
5.1.1	Goal Region Expansion	35
5.1.2	Conditional Sampling	36
5.1.3	Partial Rewiring	37
6	Long-Horizon Multi-Robot Rearrangement Planning	39
6.1	Approach	39
6.1.1	Task Space	40
7	Frameworks	41
7.1	PyBullet and Bullet	41
7.1.1	Applications	41
7.1.2	Alternatives	42
7.2	gym-pybullet-drones	42
7.3	Open Motion Planning Library	42
III	Rearrangement for Multirotor Teams	45
8	Approach	47
8.1	Problem Formulation	47

8.2	Methods and Algorithms	49
8.2.1	Overview	49
8.2.2	Planning Phase	50
8.2.3	Execution Phase	54
8.2.4	Iteration Preparation Phase	55
8.2.5	Edge Cases	55
9	Experiments	57
9.1	Scenarios	57
9.1.1	Hollow Jenga Refactored Scenario	57
9.1.2	Wall Scenario	58
9.1.3	House Scenario	58
9.1.4	Dimensions	59
9.2	Methodology	59
9.3	Results and Discussion	59
9.3.1	Task Map	60
9.3.2	Assignment Algorithm Performance	62
9.3.3	Scalability	64
10	Conclusions & Outlook	71
	Bibliography	73
	Declaration of Authorship	81

Acronyms

CHOMP Covariant Hamilton Optimization for Motion Planning

CP Convex Programming

GuSTO Guaranteed Sequential Trajectory Optimization

KOMO k-Order Markov Optimization

LBAP Linear Bottleneck Assignment Problem

LSAP Linear Sum Assignment Problem

PRM Probabilistic Roadmap Method

QP Quadratic Programming

RRT Rapidly-Exploring Random Tree

SCP Sequential Convex Programming

SCvx Successive Convexification

SLAM Simultaneous Localization and Mapping

STOMP Stochastic Trajectory Optimization for Motion Planning

TAMP Task and Motion Planning

TVBP Two-Value Boundary Problem

In 2022, around 35% of the global emissions of carbon dioxide equivalent and energy consumption originated from the largest industry in the world: the construction industry [Cdb22]. Current studies estimate that productivity has grown by only 1% in the last 20 years [BWM17]. Moreover, these argue that implementing best practices in on-site execution and technology could increase the efficiency of the construction process by up to 25%, which does not even include the implementation of new innovations. With recent advancements in general robotics, developments in automated construction picked up speed as machines became more capable of executing increasingly complex tasks.

An important role in this sector are large-scale 3D-printing robot arms that emerged to an industry of their own. Printed houses are an all-in-one solution to exploit the increased degree of automation. Moreover, it enables the seamless creation of arbitrary shapes that would pose big challenges in traditional construction. Printing a whole house is often, however, difficult to accomplish, which is why many approaches rely on printing parts and assembling them at the construction site [Nät+17].

The automation of the construction is very hard since the assembly process is subject to a number of complex rules and constraints. To implement an assembly approach in a real-world environment, it is often necessary to use multiple robots that, through parallelization and joined workspaces, can solve more complex problems faster. To tackle the assembly problem using multiple heterogeneous robots, a *Task and Motion Planning* (TAMP) approach can be employed [Har+23] that breaks down the problem into limited-horizon subproblems and finds valid task sequences to solve them. Whilst the discrete task space is used to handle the high-level rearrangement, the task actions and transitions between the tasks account for motion constraints.

Pre-printing building parts is already a flexible solution, but since steep hills are difficult to maneuver, also a more flexible assembly robot is required. Parallel to the advances in general robotics, the decrease in size of control electronics and increase in control capabilities enabled the feasibility of such simple and

lightweight aerial vehicles. Today, quadrotors are a widely spread tool with manifold professional applications due to their exceptional flexibility and simplicity. This spectrum ranges from entertainment over aerial photography to mapping and geodetic measurements and inspection of power lines [BDT17; Sha+19]. While these common applications are focused on passively collecting data, there has been a trend towards using them to actively alter the environment as well [Sha+19; Zha+22]. Here, besides the already presented advantages, the high scalability of multi-rotor teams offer unknown potential. Moreover, recent research presents multi-rotor solutions for heavy load applications of up to 20kg [Sup+17], and the Norwegian company Griff even presented an up to 225kg load multicopter that only weighs about 70kg [Sch16]. While in private use, quadrotors are mostly controlled manually, industry and research try to find ways for efficient autonomous operation. To coordinate multiple multi-rotors in a construction assembly, an integration of several methods is required.

1.1 Contribution

Based on the findings of [Har+23], we provide a proof-of-concept to apply a similar approach to aerial vehicles, namely quadrotors. We use the long-horizon TAMP problem formulation and combine it with Space-Time RRT*, an asymptotically optimal sampling-based motion planning algorithm, to plan collision-free paths for multiple agents. Further, we extend the work by robot coalition forming that enables rearranging mixed-requirements objects that a sole quadrotor can not lift. Four assignment heuristics enabling this are presented that assign objects to their target position and quadrotors to carry these objects. Finally, the algorithm is demonstrated in three construction scenarios, and the assignment algorithms are evaluated based on the total robot workload and scalability performance. The implementation was done in Python.

1.2 Methodology and Structure

This thesis is organized into three main parts. In [Part I](#), we provide the required background knowledge. In [Chapter 2](#), we introduce the motion planning problem and the most relevant approaches to how it was solved in the past. Afterward, in [Chapter 3](#), we show how motion planning algorithms can be integrated into task planning approaches, whereas [Chapter 4](#) describes the fundamental approach to

how a quadcopter is modeled and consequently controlled. [Part II](#) introduces the fundamental concepts and frameworks this thesis builds upon. In [Chapter 5](#) and [Chapter 6](#), the TAMP approach of Hartmann et al. and their space-time planner ST-RRT* are introduced. In this part's final [Chapter 7](#), we present the frameworks used for this work's practical part. Last but not least, [Part III](#) contains [Chapter 8](#) that introduces the problem formulation and our approach, whereas [Chapter 9](#) presents the scenarios of the experiments and discusses the results.

Part I

Background

The following chapter defines the motion planning problem and presents the three common basic approaches to solving it: search-based, optimization-based, and sampling-based planning.

2.1 Problem Definition

Motion planning is the process of finding a valid motion from a given start state to a given goal state while not colliding with any obstacle. To be able to understand the formal motion planning problem, some terms have to be introduced. It is essential to be familiar with the space where the planning takes place. Intuitively, this is done in a real, three-dimensional room. This representation is usable in some cases and called the workspace $\mathcal{W}$. When directly controlling robots, it is not practicable to use the workspace for planning, which is why another representation is used: the configuration space $\mathcal{C}$. The configuration space contains every possible state a robot can be in. A robot arm with two revolute joints has a two-dimensional configuration space, each dimension ranging from 0 to 2π , see [Figure 2.1](#). A single state of the robot in the configuration space is called a configuration. The motion planning problem can then be formulated as follows:

► **Definition 2.1.** Given initial and final configurations $q_{\text{start}} \in \mathcal{Q}$ and $q_{\text{goal}} \in \mathcal{Q}$, the free workspace $\mathcal{W}_{\text{free}}$, the configuration map $\mathcal{B}(\cdot)$, a cost function $J(\cdot)$, we aim to find a sequence of configurations $q : [0, 1] \rightarrow \mathcal{Q}$:

$$\begin{aligned} \arg \min_{q(p)} \quad & J(q(p)) \quad \text{s.t.} \\ & q(0) = q_{\text{start}} \\ & q(1) = q_{\text{goal}} \\ & \mathcal{B}(q(p)) \subset \mathcal{W}_{\text{free}} \quad \forall p \in [0, 1] \end{aligned} \tag{2.1}$$

Compare [\[HO22a\]](#)

There are cases where this description of the problem is not practicable to use. It is useful when the limitations of the robot are first order and can be described in configuration space. Also, this representation depends on the robot's state being able to be directly controlled by the inputs, which means the robot is required to be holonomic and not allowed to have dynamic constraints. On the other hand, when the system becomes dynamic with additional constraints, the geometric version is not sufficient to describe the problem. In this case, the system states do not directly correspond to the input, but the output is controlled by input actions, e. g. , for a primitive car, the output configuration depends on current speed and the steering wheel angle and can not be directly controlled.

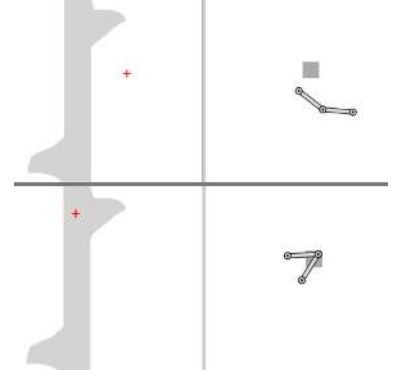


Figure 2.1: On the right: a two-joint robot in two-dimensional space with its root in the center, and a cubic obstacle nearby. On the left: the corresponding configuration space, with the red cross at the current configuration. Top: in a valid configuration, bottom: in collision [HO22c; Ort].

► **Definition 2.2.** Given initial and final configurations $q_{\text{start}} \in \mathcal{Q}$ and $q_{\text{goal}} \in \mathcal{Q}$, the free workspace $\mathcal{W}_{\text{free}}$, the configuration map $\mathcal{B}(\cdot)$, the robot dynamics f , and a cost function $J(\cdot)$, we aim to find a duration T , a sequence of controls $u : [0, T) \rightarrow \mathcal{U}$, and a sequence of configurations $q : [0, 1] \rightarrow \mathcal{Q}$:

$$\begin{aligned}
 \arg \min_{T, u(t), q(t)} \quad & J(T, u(t), q(p)) \quad \text{s.t.} \\
 & q(0) = q_{\text{start}} \\
 & q(T) = q_{\text{goal}} \\
 & \mathcal{B}(q(t)) \subset \mathcal{W}_{\text{free}} \quad \forall t \in [0, T] \\
 & \dot{q}(t) = f(q(t), u(t)) \quad \forall t \in [0, T)
 \end{aligned} \tag{2.2}$$

Compare [HO22a]



What stands out in kinodynamic planning, is that the resulting sequence of controls is bound to points in time. Whereas in geometric planning, the result is a path – a sequence of configurations – here, through the time-boundedness, the path becomes a trajectory, which is a sequence of consecutive time-configuration pairs.

2.2 Search-based Planning

Particularly relevant for solving the kinodynamic motion planning problem, search-based planning methods initially include the robots' dynamics to generate feasible solutions. Part of this approach is the discretization of the configuration space into a searchable graph called *state lattice* that can be queried with A^* and relevant variants of it. Search-based trajectory generation is one of the oldest planning methods and is still highly relevant because it provides extremely strong theoretical properties, as basic A^* and its variants are complete and optimal. [PK05] A common approach to create a state lattice uses *motion primitives*, a precalculated, static structure that describes the transition from one valid state into a neighboring one.

► **Definition 2.3.** A motion primitive is a tuple $\langle T, u(t), q(t) \rangle$, where T is the duration, $u : [0, T] \rightarrow \mathcal{U}$ the sequence of controls, and $q : [0, T] \rightarrow \mathcal{Q}$ the sequence of configurations such that:

$$\dot{q}(t) = f(q(t), u(t)) \quad \forall t \in [0, T].$$

Thus, a motion primitive is specific to a given robot type specified by the robot dynamics f [HO22d]. ◀

There are different ways to create a motion primitive: one is to apply a constant action u for a period T , another one requires a dedicated planner that calculates a feasible trajectory from a given start state to a given goal state. In the latter case, the trajectory generation is a motion planning problem itself, commonly termed *Two-Value Boundary Problem* TVBP. In order to keep the optimality property, this path between the two states must also be optimal regarding the used cost function, a process that optimization-based planners are a valid choice for (see section 2.3). This step is also responsible for ensuring that the trajectory does not violate the system constraints such as velocity, acceleration, jerk, or steering angles. By connecting multiple different motion primitives sequentially, a searchable, implicit graph can be created that contains the start- and endpoints of motion primitives as vertices and the transition between them as edges. This is typically done by formerly defining a fixed control set, which is "desired to be a minimal set of paths that, when concatenated, can re-generate any other path in the lattice" [PK05], in essence, a limited selection of motion primitives that are able to connect every reachable configuration in the discretized space. An

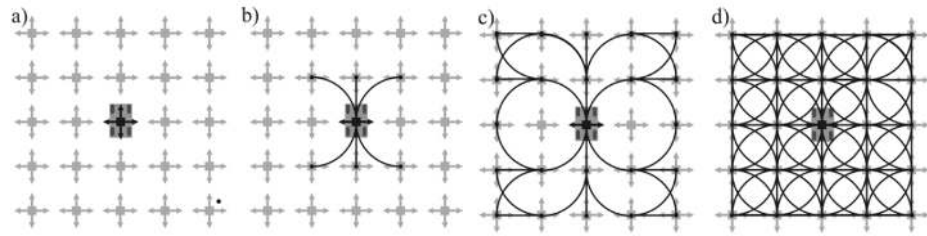


Figure 2.2: Construction Process of the State Lattice of a Reeds-Shepp Car [PK05]

example of this is picture b) in Figure 2.2, where the control set of a 2D Reeds-Shepp car contains three motion primitives for driving forward and backward each. Because of their static nature, motion primitives are a great choice for online planning, which was also confirmed by the winners of the 2007 DARPA Urban Challenge, one of the first and biggest autonomous driving contests, who used search-based methods for trajectory generation [FHL08; Kel+06]. While creating a single optimal motion primitive might result in a high computational load, this is acceptable since the computation can be done offline and has to be done only once. For quadcopters, the TVBP can even be solved online due to their differential flatness property [Liu+17].

A* Optimality A* and Anytime D*, one of the relevant A* variants, use estimates of the cost-to-go to find an optimal solution on a graph. Therefore, the performance heavily relies on the choice of the cost variable as well as the estimate, the heuristic, of it. Intuitively, the travel time from the start to the goal region is a good measure to minimize. Other options are minimization of energy consumption or distance-to-goal, whereas in practice, a combination of distance-to-goal and obstacle cost proved successful for autonomous cars [FHL08]. For quadcopters, [Liu+17] suggest two variants. A first rudimentary one where the flight time is minimized, which is able to respect velocity constraints, and because it radically simplifies the problem, it does not account for control effort and therefore often has an inaccurate and far too low estimated lower bound. Hence, the authors decided to propose another heuristic where they make educated assumptions about the goal state and solve the trajectory problem with the help of the controllability Gramian that respects optimal control. This approach improves the estimate while keeping the heuristic admissible.

Collision Avoidance Additionally, the planner must be able to meet obstacle constraints. Typically, this is done with a (possibly dynamically generated) map, where the swept volume of the motion primitives is checked against the obstacles. Depending on detected obstacles, the cost is high for trajectories close to obstacles. This obstacle cost can then be included in the heuristic of the search algorithm such that obstacle-rich ways become expensive and, as a result, impractical [FHL08].

2.3 Optimization-based Planning

Optimization-based planning algorithms tackle the motion planning problem by formulating it as an optimization problem. The target of this approach is minimizing or maximizing a given objective function while satisfying constraints. Whereas objective functions include objectives like minimization of path length or energy consumption and maximization of efficiency or smoothness, constraints might consist of obstacles for collision-free paths, kinematic/dynamic boundaries like velocity and acceleration limitations, or even motor torques.

Instead of creating a completely new path, optimization-based planners use an existing, but potentially suboptimal or infeasible trajectory as their starting point. This first trajectory is called initial guess which can be e.g. a straight line, a random sequence, or generated by another pathfinding algorithm like RRT [LaV06; Mal+21] (see Section 2.4.1), and in practical applications is often created with the help of heuristics or prior knowledge. With this trajectory as the starting point, a solution is found by improving the path by iteratively applying the optimization functions until convergence. Important to note is that, using this basic concept/in most cases, the resulting trajectory is only locally optimal, because the optimization functions converge to local minima instead of global ones. This fact underlines the importance of a high-quality initial guess. Relevant optimization-based planners are *Covariant Hamilton Optimization for Motion Planning* (CHOMP) [Rat+09], *Stochastic Trajectory Optimization for Motion Planning* (STOMP) [Kal+11], *k-order Markov Optimization* (KOMO) [Tou14] and *Sequential Convex Programming* (SCP) (see Section 2.3.2).

Because of their efficiency in complex, high-dimensional environments, algorithms like SCP are widely used choices for real-time applications like autonomous driving and the aerospace industry, whereas CHOMP and STOMP have been successfully applied for controlling robot arms. Most of these planners

directly compute trajectories instead of only computing waypoints of configurations, which makes it easier to directly apply these to non-holonomic real-world robots with kinodynamic constraints. [Mal+21]

2.3.1 Covariant Hamilton Optimization for Motion Planning

CHOMP combines kinematic feasibility and obstacle costs to generate a smooth and collision-free trajectory. While the former cost is calculated by measuring the dynamical quantities of the current trajectory, e. g. smoothness and acceleration, obstacle cost is found with the help of potential functions that are based on a precomputed signed distance field. In the optimization process, the trajectory is discretized and, depending on the cost functions, the configurations are iteratively updated using covariant gradient descent until they converge to a local optimum. [Rat+09]

2.3.2 Sequential Convex Programming

The second algorithm that should represent optimization-based planners in this thesis is Sequential Convex Programming. It is rather a general method and a class of algorithms than a single specialized one. Through its ability to not only quickly find feasible high-quality trajectories, but also to achieve this in real-time and update it with real-time information, SCP is a great choice for autonomous driving and in the aerospace industry. Just recently, a related implementation of it was used to enable the landing of SpaceX's Falcon 9 and Heavy rockets, another one to make SpaceX's Starship flip, as well as an experimental version to land the rocket New Shepard from Blue Origin. [Mal+21]

Algorithms in this class follow a given pattern [Algorithm 1](#). The key idea bases on the fact that the optimization problem is a non-convex one. As a result, solving a problem by optimization is prone to get stuck in only local minima, because gradient descent can not differentiate between deep and deepest trench but only converges towards the closest one. As a contrast, in convex functions or sets, each point can be connected to every other point without being obstructed, which consequently leads to the fact that a local minimum is always also the global one. In SCP algorithms, this core idea is used by not directly solving the optimization problem, but by splitting it into multiple convex subproblems that are convex and hence optimally solvable. This still does not necessarily result in the global optimum, because not every subproblem might contain the globally

optimal solution. However, in many real-world scenarios, this will quickly lead to good enough solutions. [Mal+21]

Algorithm 1: Sequential Convex Programming

```

BasicSCP()
1  $\bar{x}, \bar{y} = \text{InitialGuess}();$ 
2 while  $\bar{x}, \bar{y}$  not valid do
3    $\text{dyn} = \text{LinearizeDynamics}(\bar{x}, \bar{y});$ 
4    $\text{cp} = \text{ConstructConvexProblem}(\text{dyn}, \dots);$ 
5    $\bar{x}, \bar{y} = \text{Solve}(\text{cp});$ 
  
```

Relevant SCP implementations are *Successive Convexification* (SCvx) and *Guaranteed Sequential Trajectory Optimization* (GuSTO). These provide several useful properties, like the handling of nonlinear dynamics, use of arbitrary cost functions and constraints, and superlinear or for GuSTO, under certain circumstances, even quadratic, convergence rates.

2.4 Sampling-based Planning

Sampling-based planners leverage the principles of randomized sampling and can hence quickly explore high-dimensional spaces and find collision-free paths. Properties that make them a great choice for real-world scenarios. The core idea of these planners is the creation of a graph-based representation of the configuration space, where each node represents one configuration, and every edge the connection between close configurations. The configurations are generated by samplers that pick configurations from the configuration space. In a simple version, the configuration space is bounded, and a uniform sampler generates a random configuration entry for each dimension of this space. What makes sampling-based algorithms unique is how obstacle constraints are handled. Before a sample is added to the graph, it is tested for validity by checking if it is in collision. Typically, collision-checking is done by calculating the distance to all objects (or their bounding volumes) and rejecting a sample if the distance is under a threshold. This has the significant advantage that in sampling-based planning, obstacles do not have to be modeled explicitly. There are two widely

used base algorithms, *Rapidly-Exploring Random Tree* (RRT) and *Probabilistic Roadmap Method* (PRM) that share fundamental practices but differ in their concept.

2.4.1 Rapidly-Exploring Random Tree

RRT incrementally builds a tree-like structure by extending edges from the nearest neighbor of the already built tree in the direction of a randomly sampled configuration, see [Algorithm 2](#). Through an effect called Voronoi Bias, the probability of exploring yet unexplored regions is higher than the probability of extending into already explored regions. This behavior makes RRT a purely explorative and non-exploiting planner, such that it is a great choice for planning in high-dimensional spaces.

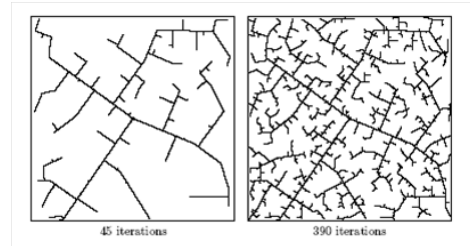


Figure 2.3: The RRT algorithm in free two-dimensional space after 45 and 390 iterations. [LaV06]

2.4.2 Probabilistic Roadmap Method

PRM consists of two stages: the construction/exploring phase and the query/exploiting phase. In the first phase, a roadmap is constructed by sampling random configurations and connecting them with an edge when their distance is below a threshold δ . As soon as a contiguous roadmap is created, the learned information can be exploited, and an arbitrary amount of paths can be queried with a shortest paths graph algorithm of choice, e. g. Dijkstra. Planners that use this two-stage approach are called multi-query planners, whereas planners that find one solution for a single start-goal pair, like RRT, are called single-query planners.

In general, the choice of algorithm depends on the problem to solve. Because of statically storing the world space in the mapping, PRM is especially effective in static environments, where many queries can be done based on the same data. In contrast, RRT is remarkably performant in higher-dimensional domains. While for PRM, lots of samples have to be generated to build a contiguous graph, due to its Voronoi Bias, RRT explores the whole space quickly and, per definition, builds a contiguous graph. Another advantage of RRT lies in its low entry point: It is fairly easy to implement, whereas PRM requires some kind of data structure to store its unused samples.

Algorithm 2: Rapidly-Exploring Random Tree [KL00]

```

BUILD_RRT( $q_{init}$ )
1  $\mathcal{T}.\text{init}(x_{init});$ 
2 for  $k = 1$  to  $K$  do
3    $q_{rand} \leftarrow \text{RANDOM\_CONFIG}();$ 
4    $\text{EXTEND}(\mathcal{T}, q_{rand});$ 
5 return  $\mathcal{T};$ 

```

```

EXTEND( $q_{init}$ )
1  $\mathcal{T}.\text{init}(x_{init});$ 
2  $q_{near} \leftarrow \text{NEAREST\_NEIGHBOR}(q_{rand}, \mathcal{T});$ 
3 if  $\text{NEW\_CONFIG}(q, q_{near}, q_{new})$  then
4    $\mathcal{T}.\text{add\_vertex}(q_{new});$ 
5    $\mathcal{T}.\text{add\_edge}(q_{near}, q_{new}, u);$ 
6   if  $q_{new} = q$  then
7     return Reached;
8   else
9     return Advanced;
10 return Trapped;

```

2.5 Sampling-based Algorithm Variants

Sampling-based planners share different problems, where each was solved by different extensions to the basic algorithm. In this section, different variants of sampling-based algorithms are presented based on the problem they solve. The main focus will be on RRT variants due to their relevance for this thesis, even though also some PRM variants are presented. Important to note is that the adaptations do not work solely standalone: many can be combined to achieve the positive impact of each solution, and depending on the problem, it will be more or less expedient.

2.5.1 RRT* and PRM*

RRT and PRM both are asymptotically complete but not asymptotically optimal. [KF11] If a solution exists, it will be found when runtime converges towards infinity. While the basic version of RRT terminates after the first solution is found, it is also possible to continue sampling e. g. until a solution is found where a certain cost might fall below a threshold. However, if the algorithm continues running without stop conditions, [KF11] has proven that the probability of obtaining a solution that is **not** optimal converges to 1. This makes the algorithm less attractive for diverse applications and lead [KF11] to develop RRT* and PRM*, respectively, algorithms that became, through different methods, asymptotically optimal.

RRT*

For RRT*, the general process is similar to the RRT one, but after connecting the sampled configuration to the nearest neighbor, instead of just adding the new configuration to the tree if collision-free, all neighbors in a given radius are considered as potential parents. Each of them has a distance or cost to the root node. Consequently, if a node exists that has a lower cost to the root node than the nearest neighbor, the initial edge is deleted from the tree, the neighbor with the lowest cost becomes the new node's parent, and the edge is appended to the graph. After the best node is found, all neighbors are checked again, if having the new node as a parent would reduce their total cost to the root node. If this is the case, they are re-parented to the new node. This process is visualized in Figure 2.4. Through this rewiring process, the tree is constantly updated to

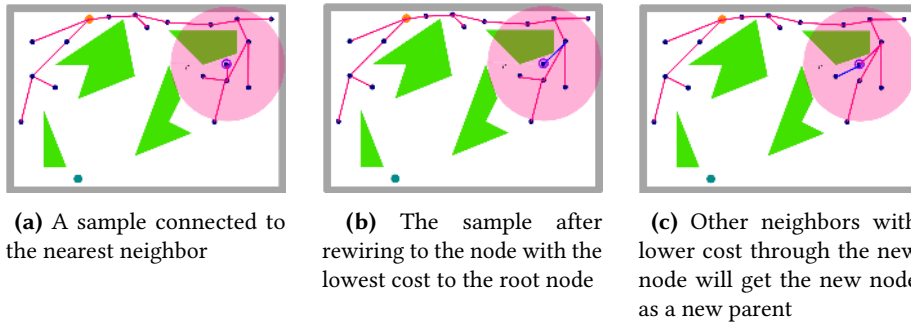


Figure 2.4: RRT* rewiring process [Hal20]

a better version and eventually, converging to infinite iterations, an optimal solution is found if it exists $\rightarrow$ asymptotical optimality. This makes this variation a highly relevant algorithm in recent motion planning algorithms. In reality, there are different methods to terminate the algorithm. One is to return the currently best solution if a path cost falls below a threshold; another one is to terminate if the solution did not improve by over $x\%$ in the last y seconds.

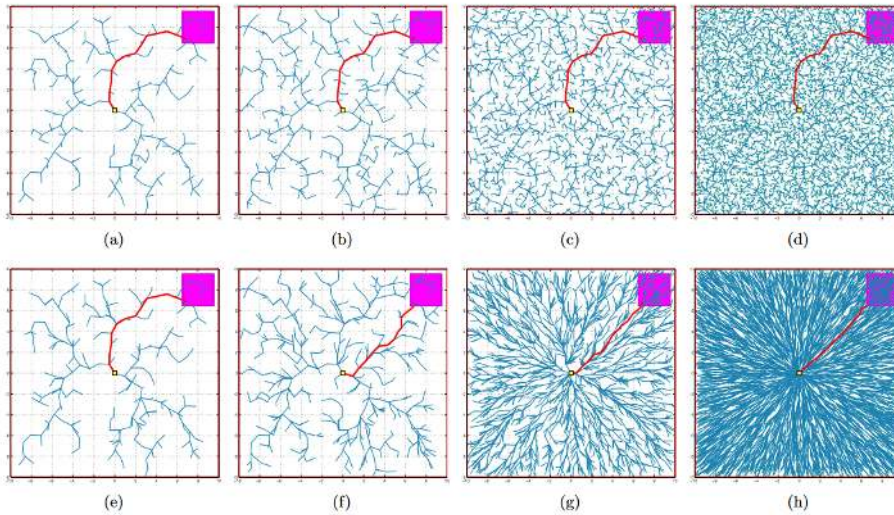


Figure 2.5: Comparison of RRT (top) and RRT* at 250, 500, 2500 and 10000 vertices, from [KF11]

PRM*

PRM* does not build on exactly base PRM, but a different version called Simple PRM (sPRM). Instead of sampling a single configuration and then trying to connect it to nearby nodes, this process is explicitly split into two parts: First, n samples are added to a set, and afterward, every node is connected to all neighbors within radius r . This algorithm is already asymptotically optimal but misses computational efficiency. What the authors in [KF11] added is a variable connection radius depending on the number of samples. With an increasing amount of samples, the radius shrinks by the rate $r = r(n) := \gamma_{\text{PRM}}(\log(n)/n)^{1/d}$, where γ_{PRM} is a constant depending on the size of free space and obstacles and d the dimension of the space. This method heavily reduces the number of edges in the graph and hence not only requires less effort while building but also when querying it.

2.5.2 RRT-Connect

For basic RRT, there exist situations called bug trap, see Figure 3.4, in which the current graph is caught inside a bug trap-like shape. This shape is quite hard to escape because, from the inside, there are almost no cases where a sample would help to extend out of the tiny entrance, and sampling within the entrance is an extremely low probability, which is shown in the picture (a) of Figure 3.4. Picture (b), on the other hand, demonstrates a second problem. To recap, Voronoi regions represent the probability of

extending from a certain node. In the presented case, these probabilities are in sum close to 1 for the regions on the outside of the bug trap and, on the contrary, near 0 for the regions within it, which means a near-zero probability of escape. Even though this planner was not designed initially to tackle this problem, it is, above all, a simple and good choice to find a path when facing it. What changes in contrast to base RRT is that it is used twice, such that two trees are built in parallel. One like the default version from the start and an additional one from the target configuration. Once the trees meet, the algorithm terminates [KL00].

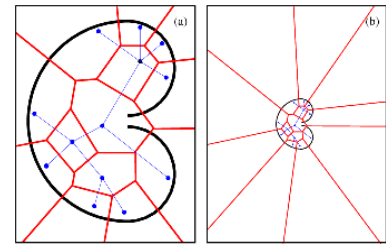


Figure 2.6: Bug traps in different sizes, with red lines representing the Voronoi region borders. [Yer+05]

This simple procedure ensures that in case of a bug trap, the algorithm does not get stuck in trying to get out of it and rather explores the space in a "probabilistic fair" manner, which makes it able to find its way into the bug trap instead of only looking for the way out.

Connect

RRT-Connect solves two problems in one: In certain environments, there is lots of free space to traverse, which RRT is not able to do very efficiently due to its purely exploratory nature. To tackle this problem, the authors of [KL00] developed a greedy method where the *EXTEND* function (see Algorithm 2), a single step in the direction of a randomly sampled configuration, is replaced by the *CONNECT* function. Instead of doing one *EXTEND* step, the *CONNECT* function calls the *EXTEND* function as long as either the sampled configuration is reached or the new sample becomes invalid, e. g. because of an obstacle. This procedure is valid, and the algorithm stays complete because each previous step configuration will have been the previous nearest neighbor of the goal configuration.

2.5.3 Goal Bias

As previously mentioned multiple times, base RRT is purely exploratory and not able to use explored information to converge to a solution quicker. To counteract this behavior and increase the exploration focus in the direction of the goal, a quite common approach is to bias the sampling. A variable $p, p \in [0, 1]$ is set, which determines the ratio of how often not a random sample, but the goal region is selected as the sample to extend. Especially in combination with the formerly introduced *CONNECT* step, this variation can decrease the runtime by directly connecting to the goal in free space [KI08].

2.5.4 Alternative Sampling Methods

As already presented in Section 2.5.2 in the form of the bug trap, sampling-based planners of every kind share one problem, which occurs when the only valid path between start and goal crosses a "low-probability zone" that is commonly called "narrow passage". The two-dimensional case is shown in Figure 3.4. Due to the asymptotic completeness properties of RRT and PRM, in theory, eventually,

a path will be found in any case, but in practice, the passage might be so small that the probability of finding a path becomes practically zero.

To tackle this problem, the authors of [BOV99; Hsu+03] have come up with two new sampling methods. During common sampling techniques, lots of information is lost because invalid samples are rejected and immediately forgotten. [BOV99; Hsu+03] changed this by also leveraging the information of invalid samples. The target of the new methods is to find samples in regions that are otherwise hard to reach. The presented methods include sampling methods that sample close to and between obstacles, which can help to find samples within a narrow passage. Note that these samplers, if used singularly, might render the algorithm incomplete. It is more useful to include them similarly to the goal bias to sample a specific ratio of the samples in one of the ways.

Gaussian Sampling

After a sample is known to be invalid, in Gaussian Sampling, an N-dimensional Gaussian function is created with its mean μ at the previously sampled invalid configuration. Thereafter, a second sample is generated, but not uniformly as before, but from the newly created Gaussian distribution. If this second sample is valid, it is guaranteed to be close to an obstacle. Otherwise, the sample is rejected, and the process restarts. [BOV99]

Bridge Sampling

Bridge Sampling is even more specific. Here, the first failing sample is also overlaid with a Gaussian being sampled from afterward. Contrary to Gaussian Sampling, the second sample has to be invalid to successfully continue the process. If this is the case, the midpoint between the samples is again tested for clearance. If it is valid, it is accepted and guaranteed to be located between two "walls", e. g. in a narrow passage or in a corner. If not, it is rejected, and the process restarts [Hsu+03].

2.5.5 LazyPRM

Whilst PRM reliably produces solutions of reasonable quality, in practice, it is computationally costly to perform collision checking for all sampled nodes [BK00]. PRM variant Lazy PRM improves the performance by reducing the

number of validity checks to a minimum by only checking configurations that are part of a potential solution path. In [BK00], the direct comparison between Lazy PRM and PRM showed that Lazy PRM explores less than 0.1% of the solution space (versus 100% for PRM), the average planning time was reduced by over 99% (71s versus 35840s for PRM in the worst scenario) and 26% of all collision checks were part of a solution path and therefore inevitable. These properties make Lazy PRM a great choice for real-world scenarios.

2.5.6 Optimality

The following Table 2.5.6 contains the different presented sampling algorithm variants with their algorithmic properties. Because of their random sampling nature, all of them find a path if it exists. Due to the inability to terminate if no solution exists, the algorithms are only classified as probabilistically complete. While RRT was proven to find a suboptimal solution with a probability of 1, rewiring methods approach the optimal solution with time running against infinity and hence are asymptotically optimal.

Algorithm	Probabilistic Completeness	Asymptotic Optimality
PRM	Yes	No
RRT	Yes	No
PRM*	Yes	Yes
RRT*	Yes	Yes
Lazy PRM	Yes	No
Lazy PRM*	Yes	Yes
RRT-Connect	Yes	No

Table 2.1: Sampling Based Variants and their completeness and optimality consideration. [Cho23; KF11]

2.6 Comparison

The different planning approaches have different ad- and disadvantages. Search-based methods, with A* and its variants, provide a (depending on the space resolution) optimal approach to solving the motion planning problem. While optimization-based algorithms can quickly find solutions in high-dimensional

spaces, advanced sampling-based approaches stand out in finding globally optimal solutions in complex environments where explicitly modeling the obstacles is challenging. The following table gives an overview of and comments on the provided properties.

	Search-Based	Optimization-Based	Sampling-Based
Completeness	resolution-complete (A^*)	No (SCP), Yes (CP)	probabilistically complete (RRT*)
Optimality	optimal w.r.t. resolution (A^*)	locally optimal (SCP), globally optimal (CP)	asymptotically optimal (RRT*)
Complexity/ Convergence	polynomial	polynomial time to find optimal solution (LP, some QP)	exponential convergence for any solution; sublinear to optimal solution (PRM*)
Scalability with state/ action dimension	exponential	polynomial	exponential
Scalability with duration/ path length	almost linear (heuristic dependent)	polynomial	$\sim$ linear

Table 2.2: Sampling Based Variants and their algorithmic properties [HO22b].
(CP: Convex Programming, LP: Linear Programming, QP: Quadratic Programming)

3

Task and Motion Planning

The core idea of task and motion planning is adding a discrete high-level task dimension to the planning process, enabling agents to reach high-level goals. The following section introduces the fundamentals of this idea.

3.1 Discrete Task Planning

Task planning is a frequently used term in artificial intelligence research, where discrete tasks are used to plan for and describe the current state of autonomous agents. Formally, the simplest version of the AI planning problem is the specification of states $\mathcal{S}$, valid transitions between the states $\mathcal{T} \subseteq \mathcal{S} \times \mathcal{S}$, a start state $s_0 \in \mathcal{S}$ and a set of goal states $\mathcal{S}_* \subseteq \mathcal{S}$. To change from one state into the next, transitions $t = \langle s, s' \rangle \in \mathcal{T}$ are used. The planning objective is finding a plan π that contains a sequence from start state s_0 to goal state $s_* \in \mathcal{S}_*$ [Gar+20]. In practice, state and transitions can look like the following (shortened) example in Figure 3.1, see also Figure 3.2 for the full scenario:

Example State:

```
{atRob=Plate, holding=None, at[Book]=Table, at[Pizza]=Box}
```

Example Transitions:

```
moveF[loc1, loc2]
  pre: atRob = loc1, holding = None
  eff: atRob ← loc2
pick[obj, loc]
  pre: atRob = loc, holding = None, at[obj] = loc
  eff: at[obj] ← None, holding ← obj
```

Figure 3.1: Example state and transitions for a robot [Gar+20]

They are pretty practical. The presented state describes the robot being at position Plate, not holding anything, and the positions of the book and the pizza. In

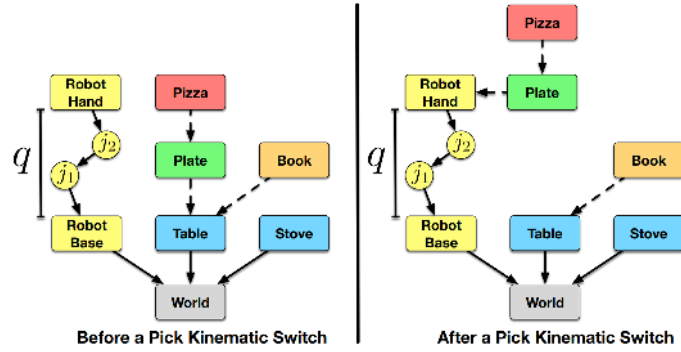


Figure 3.2: Example of a kinematic tree [Gar+20]

the transitions, **pre** describes the precondition to start the transition, whereas **eff** describes the effect that this transition has on the system. **moveF** is a transition to move a robot arm without any object in its claws, **pick** is a transition that is started when the robot is at a given position *loc*, holding nothing and the object is also at position *loc*.

We leverage this discretization's advantages to find task sequences that solve subtasks until the supertask is solved.

3.1.1 Rearrangement Planning

In this thesis, the supertask is to solve an assembly or rearrangement problem: find a sequence of motions to rearrange a set of given objects to assemble a final product [HLW00].

Kinematic Trees

Because multiple objects of type *t* exist, several sequences generate the same final product. To model changes in the environment, confirm if an assembly is the final one, and verify transitions between tasks, it is helpful to find a representation model for the environment state. One way to achieve this is kinematic trees. Kinematic trees are graph representations of object dependencies where the vertices are the objects in the scene, and the edges describe attachments between objects and agents [Gar+20]. Figure 3.2 shows two kinematic trees of a kitchen scenario where a two-joint robot picks up a plate with pizza.

3.2 Task and Motion Planning

To add motion planning to the formulation, the transitions (e. g. Figure 3.1) are extended by motion planning constraints. The introduced constraints enforce start and goal configurations and the movement between these configurations in the given mode (including the mode's constraints), where a mode is a potential object transported during the movement. The following Figure 3.3 shows, again simplified, how the example transitions from Figure 3.1 can look when motion planning constraints are added [Gar+20].

Example Transitions:

```

moveF( $q, \tau, q'$ )
  con:  $\tau(0) = q, \tau(1) = q', (\forall t \in [0, 1] : \tau(t) \in C_{free})$ 
  pre:  $atRob = q$ 
  eff:  $atRob \leftarrow q'$ 
pick[obj]( $q, p, g$ )
  con:  $Kin[obj](q, p, g)$ 
  pre:  $atRob = q, holding = None, at[obj] = p$ 
  eff:  $at[obj] \leftarrow g, holding \leftarrow obj$ 

```

Figure 3.3: Example state and transitions for a robot. For transition `pick`, a constraint `Kin` is introduced that enforces when robot at pose q holds object `obj` in grasp g , that `obj` will be at pose p .

In some instances, it is helpful to use a fixed series of transitions that follow each other. When we add the transitions `moveH` (move while holding an object) and `release` to previous examples, Such a sequence might look like the following, where `o` is an arbitrary object selected just for this sequence:

$$\pi = [\text{moveF}(q_0, \tau_1, q_1), \text{pick}[o](q_1, p_0, g_2), \\ \text{moveH}[o](q_1, \tau_3, q_3), \text{release}[o](q_3, p_4, g_2)]$$

Such a sequence is called *plan skeleton* [Gar+20].

3.2.1 Multi Robot Task Planning

To apply this problem to multi-robot long-horizon planning, multiple tasks or plan skeletons must be unambiguously assigned to agents in multiple sequences. To additionally solve the assembly problem, not only assigning tasks to agents is

required, but also assigning objects of different types to their final position. These problems can be generalized under the umbrella of the assignment problem.

The assignment problem

"Assignment problems deal with the question of how to assign n items (jobs, students) to n other items (machines, tasks)." [BDM09]. First of all, this problem can be solved arbitrarily. A simple approach is a random assignment where every student $u \in U$ is randomly assigned to a task $v \in V$. Depending on the objective (cost function), there are optimal ways to solve this problem. One objective might be to reduce the total cost of all assignments, and another to minimize the most expensive assignment. The former class is called *Linear Sum Assignment Problem* (LSAP) and the latter *Linear Bottleneck Assignment Problem* (LBAP). Due to its relevance in linear programming and this thesis, we will only further introduce the linear sum assignment problem.

For the manifold ways to represent assignment problems (matrix, bipartite graph, flow graph, ...), optimal, domain-dependent ways to solve the LSAP were found. One of the first exponential-time methods to solve the problem on a bipartite graph is the Hungarian algorithm.

The Hungarian algorithm is a linear optimization technique that uses augmenting paths in a bipartite graph. It is a two-stage method that starts with a feasible partial solution generated from the cost matrix of every available assignment by subtracting the minimum of every row from every row value and then doing the same for each column. It then iteratively seeks augmenting paths in the bipartite graph and tries to increase the cardinality of the current assignment until all assignments with cost 0 are found (each subtraction of the minimum value in rows and columns of the cost matrix creates new zeroes). When, at this point, all vertices are assigned, the optimal solution is found. Otherwise, again, the minimum values in rows and columns are subtracted from each value, and the augmented path is grown, which is repeated until all vertices are assigned [BDM09].

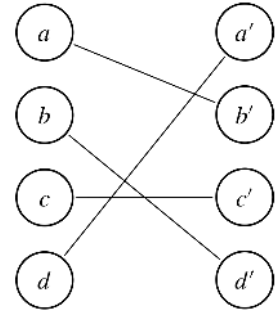


Figure 3.4: Example for an assignment represented by a bipartite graph [BDM09]

4

Controlling a Quadcopter

The following chapter introduces the quadrotor as a mathematical model and how it can be actuated. The first part presents a simplified mathematical model for quadrotor dynamics, and the second part provides a brief introduction to controllers and an example of how a real-world quadrotor, a Bitcraze Crazyflie 2.0, is controlled for a stable flight.

4.1 Quadcopter Dynamics

For quadcopters, the high degree of freedom ($6, \mathbb{R}^3 \times SO(3) = SE(3)$) has its costs. Due to having only four inputs, the system is heavily underactuated, such that in quadrotor control, translation and rotation are combined, which results in highly nonlinear dynamics, especially when, additionally, aerodynamic effects are taken into account. A commonly used simplified dynamic model is based on the Newton-Euler equations from classical mechanics that describe the translation and rotation of a rigid body that forces are applied to. Before introducing them, we have to break down the required components. As the rigid body, we define the model as simple as possible: a cross of two equally long "sticks" of length $2l$, where l is the arm length with the angle $\delta = \pi/2$ between them, see Figure 4.1. At the intersection of these sticks is the center of mass and the origin for determining the quadcopter position, whereas the effects of motor rotations are acting on the ends of the sticks. The target is to find a model that determines the change in state $\dot{x}(\omega_i) = [\dot{p}(\omega_i), \dot{q}(\omega_i), \dot{v}(\omega_i), \dot{\omega}_B(\omega_i)]^T$ from the

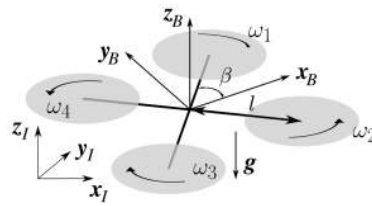


Figure 4.1: Simplified model of a quadrotor [Sun+22]

current state $x(t) = [p(t), q(t), v(t), \omega_B(t)]^T$ with position $p(t) \in \mathbb{R}^3$, rotation $q(t) \in \mathbb{H}$, velocity $v(t) \in \mathbb{R}^3$ and angular velocity $\omega_B(t) \in \mathbb{R}^3$ when rotor speeds ω_i are applied. The rotation is represented as a quaternion. This has multiple advantages; it solves the gimbal lock problem that the Euler angle representation has [Qua17] and is numerically more stable than cartesian rotation matrices [Boy17].

The force on the motor positions is defined by the squared rotation speed ω_i and the thrust coefficient C_T that depends, among other things, on the propellor radius and density of the air [LN16].

$$f_i = C_T \omega_i^2 \quad (4.1)$$

Hence, we can define the total thrust vector

$$f_T = \begin{pmatrix} 0 \\ 0 \\ C_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2) \end{pmatrix} \quad (4.2)$$

and the momentum τ , which takes into account the origin of the acting force. With β_i being the angle between the quadcopter arm and the x-axis x_B , C_D the torque drag coefficient, and α_i the rotation direction of the motor:

$$\tau = \sum_{i=1}^4 P_i \times f_i = \begin{pmatrix} l \sum_i^4 \sin(\beta_i) f_i \\ l \sum_i^4 \cos(\beta_i) f_i \\ C_D \sum_i^4 \alpha_i f_i \end{pmatrix} \quad (4.3)$$

It is important to note that $\alpha_i \in \{-1, 1\}$. The reason is the rotations of the body in the opposite direction to the rotation of a motor that arises due to the conservation of angular momentum. To counteract this effect, the amount of motors turning clockwise should equal the number of motors turning counter-clockwise, alternatingly distributed.

The thrust of the propellers always points in the z-direction of the body. Because we want to find a model for the change in state from the global perspective, the current formulation is not sufficient. For a transparent calculation, we introduce the cartesian *inertial (world) frame* $\mathcal{F}_I : \{x_I, y_I, z_I\}$ and the cartesian *body frame* $\mathcal{F}_B : \{x_B, y_B, z_B\}$, see Figure 4.1. As mentioned, thrust is always pointing in the z-direction in the body frame. In the inertial frame, the same holds for gravity g .

Until now, the quantities were expressed in the body frame, but to represent global changes, we must transform the thrust from the body frame into the global frame. The transformation (of rotations) between the two coordinate frames is done with the help of the well-known rotation matrix $\mathcal{R}(q)$, similar to [FN13]. The last thing we want to introduce is the inertia matrix that describes how a body reacts to applied torque in either direction. Due to the strong symmetry properties of the body, all of the crossed terms in the inertia matrix are zero:

$$\mathbf{J} = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix} \quad (4.4)$$

With the resulting Newton-Euler equation, where m and I_3 denote the mass and three-dimensional unit matrix,

$$\begin{pmatrix} \mathcal{R}(q)f_T \\ \tau \end{pmatrix} = \begin{pmatrix} mI_3 & 0 \\ 0 & \mathbf{J} \end{pmatrix} \begin{pmatrix} \dot{v} \\ \dot{\omega}_B \end{pmatrix} + \begin{pmatrix} -g \\ \omega_B \times \mathbf{J}\omega_B \end{pmatrix} \quad (4.5)$$

we find

$$\dot{v} = \frac{1}{m}\mathcal{R}(q)f_T + g \quad (4.6) \quad \dot{\omega}_B = \mathbf{J}^{-1}(\tau - \omega_B \times \mathbf{J}\omega_B) \quad (4.7)$$

The following equations represent the position and rotational change [FN13; Sun+22], where $\otimes$ denotes the quaternion product:

$$\dot{p} = v \quad (4.8) \quad \dot{q} = \frac{1}{2}q \otimes \begin{pmatrix} 0 \\ \omega_B \end{pmatrix} \quad (4.9)$$

4.2 Controllers

The dynamics can be used to design controllers that enable following a trajectory. Without looking at the inside, for our use case, a controller is a function that iteratively receives elements from a trajectory and the current system state as input and, at each step, returns the necessary actions to compensate for the difference between the current and the required state. There are three basic building blocks a controller can consist of: the proportional, the integrator, and the differential element.

4.2.1 Basic Building Blocks

Their names describe how these elements deal with the error (the difference between actual and required value), and each solves a problem of how the system is controlled. The proportional element multiplies the error by a value k_p and is responsible for how strong the system reacts to an error. Because we multiply the error with an arbitrary value $\neq 1$, there will be a permanent offset when using only a proportional element. To compensate for this, the integrator sums up past errors multiplied by coefficient k_i and can hence eliminate such permanent offset. Finally, the differential element tracks how fast the error changes and is an excellent tool to dampen the system. The differential error is scaled by coefficient k_d . These basic building blocks can be combined and tuned (by changing the coefficients k_p , k_i , and k_d) to deliver stable and high-quality output (but this is a thesis in itself [LN16]). Common combinations are PI, PD, and PID controllers, where the tradeoff is complexity, tuning effort, and available computation power versus high-quality results.

4.2.2 Control of the Bitcraze Crazyflie

In the following, we roughly introduce the PID controller structure that is part of the firmware (version 2023.07) of the Bitcraze Crazyflie 2.1, the quadcopter used in simulation in this thesis. Figure 4.2 shows a cascaded control struc-

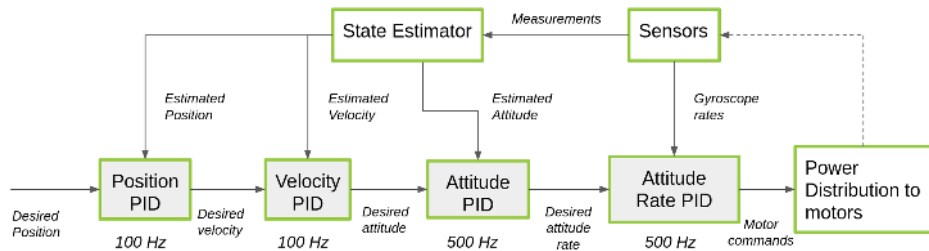


Figure 4.2: PID Structure in the Bitcraze Crazyflie Firmware [Bit23a]

ture that can be separated in two parts with different control frequencies: The position and velocity loop that runs 100 times per second and the attitude (orientation) loop that runs 500 times per second. Due to the essential role of

attitude in the system's stability, the attitude loop runs much more often. The state estimator estimates the real drone's current position, velocity, attitude, and attitude rate (change in attitude). In simulation environments like used in this thesis, this information is provided by the physics simulator. The first part of the cascade is initiated with the external input of the desired system state. An arbitrary set of state set variables can be input depending on the user requirements. For the whole experience, we assume the input set value to be a position vector $p_{desired} = [x_{desired}, y_{desired}, z_{desired}]^T \in \mathbb{R}^3$. In "Position PID", we calculate the desired velocity for the movement of $p_{current}$ to $p_{desired}$ in timestep $PD (= 1/\text{frequency}_{pos_control})$. While doing so, the errors are multiplied with their designated coefficient k_{px} , k_{py} and k_{pz} respectively. This result is used to calculate the desired pitch and roll attitude in "Velocity PID" that allows this movement, similar to the velocity calculation, as well as the desired thrust for the rigid body, which is the desired velocity in z-direction. These desired values are then the inputs for "Attitude PID", the first controller in the faster loop. Here, the desired attitude rate is calculated, then used to calculate the attitude error in "Attitude Rate PID", that together with the thrust will be output to the motor controllers. For each motor, those values are converted to the desired PWM that is then applied to the power hardware that controls the motor [Bit23b; Bit23c].

Part II

Fundamental Building Blocks

In the paper "ST-RRT*: Asymptotically-Optimal Bidirectional Motion Planning through Space-Time" [Gro+22], the authors present a novel sampling-based motion planning algorithm for planning through space-time. The algorithm, ST-RRT*, builds on RRT-Connect by utilizing the *CONNECT* step, bidirectional trees, and goal bias, uses rewiring similar to RRT*, and enables planning in dynamic environments. It does so by adding a time dimension to the state space, assuming knowledge over all already planned paths, and leveraging this information with its time-respecting collision checker. Another feature is the permission for unknown arrival times and the ability to operate in an unbounded time-space, making planning in dynamic environments even more flexible. The algorithm is probabilistically complete and asymptotically optimal regarding the shortest arrival time. The following pseudocode [Algorithm 1](#) from [Har+23] is a simplified version of the algorithm that excludes for the top-level understanding required variables.

5.1 Features and Challenges

Finding proper time boundaries for goal dimensions has been problematic in other such dynamic full-knowledge planning algorithms (Time-Based RRT, Safe Interval Planning). ST-RRT* tackles this problem by using dynamic time boundaries. The algorithm begins by estimating a feasible initial maximal goal time and then starts to sample. If after n samples, no solution is found, the time bound is increased, and the larger space is searched, which is repeated until a solution is found. This iteratively increasing time bound is also the reason for the ability of the algorithm to operate in unbounded time-space. Here, an interesting problem came up.

5.1.1 Goal Region Expansion

When dynamically changing the size of a previously sampled space, unified sampling becomes unfair because the samples of the first time bound are kept,

Algorithm 3: High-Level ST-RRT* [Har+23]

```

1 BUILD-ST-RRT*( $x_0$ , goal_sampler)
  while not stopped do
2    $t_{lb}, t_{ub} \leftarrow \text{update\_bounds}()$ ;
3   if  $\text{rnd}(0, 1) < p_{\text{goal}}$  then
4      $t \leftarrow \text{sample}(t_{lb}, t_{ub})$ ;
5      $q_g \leftarrow \text{goal\_sampler}(t)$ ;
6      $\text{add\_goal}((t, q_g))$ ;
7    $q \leftarrow \text{sample\_valid\_state}()$ ;
8    $t \leftarrow \text{sample\_valid\_time}(q)$ ;
9   if not  $x_{\text{new}} \leftarrow \text{EXTEND}(t, q, T_a) = \text{trapped}$  then
10    if  $\text{CONNECT}(x_{\text{new}}, T_b) = \text{reached}$  then
11      return  $\text{extract\_path}()$ ;
12     $\text{swap}(T_a, T_b)$ ;
13 return  $\emptyset$ ;

```

and samples will not be equally distributed anymore. Sampling only the new space (for a fixed amount of samples, as before) is also not valid because it might have a different size, and we also want to keep discovering the previous one. The result is the calculation of a batch probability that relies on the space increase factor and a user-specified ratio. Consequently, depending on a random value, one of the two batches is sampled, and fairness as good as the user-selected variables is established.

5.1.2 Conditional Sampling

Another advantage of the created sampling method is the ability to implicitly model velocity constraints. This is done by sampling a configuration q from the state space and afterward a time from the range of possible times, whose bounds are calculated from the minimal arrival time of the initial config and the user-defined maximal velocity.

5.1.3 Partial Rewiring

In a bidirectional time tree, the efficiency of the start trees can never influence the total arrival time because it is solely influenced by the best time in the goal tree. They use this fact to introduce only a partial rewiring step that solely rewires goal trees. As rewiring costs, the time in each node is considered.

6

Long-Horizon Multi-Robot Rearrangement Planning

The work in [Har+23] builds the basis for this thesis and made curious for more research and a POC for a similar system using quadcopters. They try to solve the following problem:

"Given n unique objects $o \in \mathcal{O} [\dots]$, with initial poses $p_{o,init} \in SE(3)$ at time $t = 0$, and m heterogeneous robots $r \in R [\dots]$, the aim is to *rearrange* all objects to their (given) goal locations $p_{o,goal} \in SE(3) [\dots]$ " [Har+23].

6.1 Approach

The problem is solved by iteratively decomposing the assembly problem into discrete limited-horizon subproblems. These subproblems are sequenced with the help of time-embedded *keyframes* (constrained transitions from one task into the next that are optimization problems themselves). These keyframes determine the start and end configurations for the motion planning step that eventually connects the keyframes of each robot with a path in the space-time state space using ST-RRT*. Figure 6.1 shows a high-level perspective of the process. What is remarkable about this work is that it supports heterogeneous

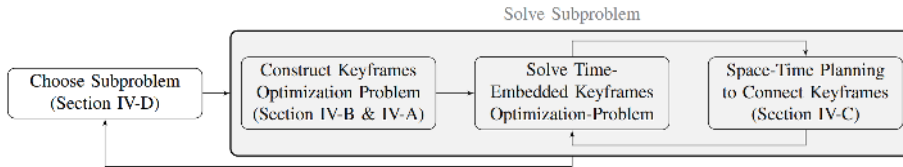


Figure 6.1: Top-level view on the methods used in this work [Har+23]

manipulators with unique configuration spaces. In the provided examples, there were cases where the 6-DOF robot arms could not place an object because its desired position exceeded the available z-limit. To bypass this problem, the authors utilized a second robot class, a crane, that can lift objects higher but

also creates the necessity for a handover from a robot arm to the crane, posing an additional challenge. Aside from that, it is noteworthy how completeness is achieved. For each placeable object, multiple task sequences for multiple robots are calculated. If for none of the sequences, a solution is found, the infeasibility must stem from previously placed objects. In such a case, the current state is backtracked, and the previous placement of objects is rewound until a solution is found.

6.1.1 Task Space

With the requirement for handover introduced, these are the available tasks a robot can execute: `pick`, `place`, `retract`, and `handover`. Except for `retract`, these are self-explanatory. `retract` is a task that became necessary because, often, unassigned robots that had just placed an object blocked the target area, and, hence, no solution was found. In consequence, `retract` is a task that lets the robot exit the building area to a random position in the scene.

The following section introduces tools that were vital to accomplish the implementation of this work. These are the physics simulator *PyBullet*, a framework that integrates PyBullet and multi-robot flight, *gym-pybullet-drones*, and the *Open Motion Planning Library* (OMPL).

7.1 PyBullet and Bullet

PyBullet is a Python module for robotics simulation and machine learning. It contains Python bindings that call the underlying Bullet C-API, an interface to the open-source C Bullet physics and render engine. Bullets core functionalities include forward dynamics simulation, inverse dynamics computation, forward and inverse kinematics, and collision detection, which make it an excellent real-time physics simulator [CB16].

7.1.1 Applications

PyBullet and the basic Bullet C API are used in various research projects. Habitat Lab, a Meta-originated research simulation platform for training embodied AI, uses the Bullet physics engine to simulate physics in their various 3D simulation environments. Because of their focus on training quantity and, therefore, only the essential functionalities of such a training platform, Habitat 2.0 Sim promises to reach up to over 25.000 simulation steps per second, which makes it about 850× faster than real-time and up to 100× faster than relevant competitors [Szo+22]. These numbers not only underline the relevance of a potent pipeline, including a fast physics simulator in such an application but also demonstrate the actual performance and capabilities of the Bullet physics engine. Also, PyBullet as a python module finds lots of attention in research [Col+21]: it enabled algorithms to learn the locomotion skills of animals by imitation [Bin+20] and helped to teach robots movements of assisting tasks in nursing professions [Eri+19].

7.1.2 Alternatives

There are several alternative choices when it comes to professional robotic simulations. Besides PyBullet, Gazebo [KH04], Coppeliasim [RSF13], and Webots [Mic04] are popular tools [Col+21] that were considered for this thesis, each of them having their reason of existence in their field. Most of these frameworks are standalone software supporting ROS integration and mainly focused on ground applications. Even though they included examples of quadcopter control and flight, setting up simple motion planning tasks proved unintuitive and clunky. Especially for PyBullet, with its lightweight, easy-to-integrate Python API, they were no match. On top, a PyBullet-based quadcopter training platform with large parts of quadcopter control and multi-robot simulation is already implemented: `gym-pybullet-drones` [Pan+21].

7.2 `gym-pybullet-drones`

Similar to the mentioned Habitat Lab, `gym-pybullet-drones` is a Gymnasium-based framework for quadcopter reinforcement learning, where, due to its popularity in the field, the nanocopter Crazyflie 2 plays a central role. Gymnasium is an open-source meta Python library for developing reinforcement learning algorithms and provides environments and examples for this purpose. In its fundamental intent, `gym-pybullet-drones` should be used to train reinforcement algorithms for teams of multi-rotors, whereas, for this thesis, its environments, controllers, and codebase played a crucial role as the first project building blocks, mainly because it already unites PyBullet and quadcopter control.

7.3 Open Motion Planning Library

When doing motion planning projects, there is no way around the *Open Motion Planning Library* (OMPL) [ŠMK12]. It is an open-source C++ motion planning framework that includes many of the well-known sampling-based planning algorithms, as well as essential structures like state spaces and their corresponding samplers. By intent, it does not enforce frontend and collision checking capabilities, such that these can be chosen depending on the project's needs. These properties make OMPL an excellent choice for fast prototyping (especially with the available Python bindings), larger motion planning projects, and the

development of custom planners. Also, the planner that this thesis heavily relies on, ST-RRT*, including the required spaces, was already implemented in OMPL by its authors [[Gro+22](#)].

Part III

Rearrangement for Multicopter Teams

In this chapter, the approach will be introduced. We begin by defining the multi-agent long-horizon rearrangement problem and the assumptions that were made to be able to solve it. Thereafter, the applied methods are presented, first from a high-level point of view and then with a continually increasing level of detail.

8.1 Problem Formulation

Adapted from [Har+23], we try to solve the following problem: Given n objects of m different types, indexed by $o \in O$, $|O| = n$, with initial poses $p_o^0 \in SE(3)$ at time $t = 0$, and j quadrotors, indexed by $r \in R$, $|R| = j$, the aim is to rearrange the objects, such that every given goal location $p_o^G \in SE(3)$ is occupied by an object of the required type τ_m . All quadrotors share the same configuration space $X \subset \mathbb{R}^3 \times SO(2)$ (A definition that can be relaxed to $Q_r \subset \mathbb{R}^{d_r}$ when other robot types are used).

The problem is formulated as a non-linear mathematical program over the path $x : [0, T] \rightarrow X$. The configuration space serves as a space for quadrotors as well as objects. Depending on the currently available task, different constraints are active. For instance, after a `pick` task, the object has to be attached to the robot. The active constraints are determined by the current task assignment $s \in \mathcal{S} = S_{r_1} \times \dots \times S_{r_m}$ where S_r indicates the feasible tasks for robot r . $s_{r,1:K_r} \in \mathbb{S}(\mathcal{R}, \mathcal{O})$ denotes the discrete sequence of tasks of robot r , with $s_{r,j} \in S_r$ and K_r being the amount of states in the sequence. $\mathbb{S}(\mathcal{R}, \mathcal{O})$ is the set of valid state sequences for robots $\mathcal{R}$ and objects $\mathcal{O}$ connected by first-order logic-language, similar to Figure 3.1. In this case, this set is strongly reduced thanks to the use of a fixed plan skeleton π (see Section 8.2.2). Traditionally, TAMP task switches happen at fixed intervals. The presented problem formulation allows task assignment switches at any point in time. The reason for this is the implicit *scheduling function* $k : [0, T] \rightarrow \mathcal{N} = (1, \dots, K_1) \times \dots \times (1, \dots, K_m)$ that allows accessing the index of each task s at time t such that $s(t) = s_{k(t)}$ and is constrained to maintain the index order, e. g. $k_r(t_1) \leq k_r(t_2), \forall t_1 \leq t_2$.

Summarizing, we try to find the path x , the terminal time $T > 0$, the scheduling function k and the sequences of discrete states $\{s_{r,1:K_r}\}_{r=1}^m$ to optimize

$$\min_{x,T,k,\{s_{r,1:K_r}\}_{r=1}^m} \int_0^T c(x(t), \dot{x}(t), \ddot{x}(t)) dt \quad (8.1a)$$

$$\text{s.t.} \quad x(0) = x_0 \quad (8.1b)$$

$$\forall t \in [0, T] : \quad g(x(t), \dot{x}(t), s_{k(t)}) \leq 0 \quad (8.1c)$$

$$\{s_{r,1:K_r}\}_{r=1}^m \in \mathbb{S}(\mathcal{S}, \mathcal{O}) \quad (8.1d)$$

$$g_{\text{goal}}(x(T), p_{\mathcal{O}}^G, \mathcal{O}) \leq 0. \quad (8.1e)$$

Here, c is the cost function to be minimized and can be flight time, path length, control cost, or 0 if only a feasible solution is to be found. In (8.1c), $s_{k(t)}$ describes all constraints of active tasks at time t using constraint function g , where $s_{k(t)}$ is a set of task constraints containing each robots' r current task: $s_{r,k_r(t)} \in \mathcal{S}_r$ at time t . Here, also the collision constraints will be introduced. Equations (8.1b) and (8.1e) introduce the initial and goal position constraints. While the former enforces that quadcopters and objects are at their defined initial position, the latter that every goal position has to be occupied by an object.

The following assumptions are made:

Known world state We know the initial poses of quadcopters and objects, the desired final poses of the subset of required objects to build the final state, and all intermediate robot and object configurations at any point in time, which enables the sampling of valid configurations.

Monotonic rearrangement Traditional rearrangement planning allows objects to be concealed by others, such that the desired object has to be uncovered by removing the blocking one. We do not allow such situations by assuming that each object is handled a single time.

No grasp dynamics The quadcopters do not have an actual grasp mechanism (gripper or similar). The attachment of quadcopters to objects is done by joining the URDFs of the respective participants and, depending on the object type, modeled by a fixed or spherical joint.

Simple quadcopter dynamics For simulation purposes, a simple model similar to the one presented in [Section 4.1](#) is used. There are no rotational drag forces, no wind, and no blade flapping (propellor deformation at high rotation speeds with flexible material) acting.

Low-dimensional space planning Because no special quadcopter maneuvers are required for the given use-case, planning takes place for the three-dimensional position and the yaw of the quadcopter $\mathbb{R}^3 \times SO(2)$.

8.2 Methods and Algorithms

The presented problem is practically not solvable using a global strategy. The presented approach decomposes the problem into simpler subproblems and solves them such that the sum of solved subproblems constitutes a feasible solution to the original problem. The following section describes the top-level process and gives a detailed overview of its relevant substeps.

8.2.1 Overview

The high-level process can be separated into three phases that are repeated until the final tree is a subset of the current tree:

1. **Planning Phase (TAMP)**, see [Section 8.2.2](#)
 - a) Select the subproblems: Assign target positions to available objects and available quadcopters to these object and to final position assignments
 - b) Generate plan skeletons for each assigned quadcopter
2. **Execution Phase (ST-RRT*)**, see [Section 8.2.3](#)
 - a) If possible, start the next task of all drones (dependent on constraints and preconditions)
3. **Iteration Preparation Phase**, see [Section 8.2.4](#)
 - a) Update the global time depending on the next completed task
 - b) Update quadcopter state dependent on the updated time
 - c) Synchronize the world state

Algorithm 4 shows the detailed pseudocode of this process.

Algorithm 4: TAMP

```

SOLVE (current_tree,  $\mathcal{R}$ )
1  $t \leftarrow 0$   $s_{\mathcal{R}} \leftarrow \emptyset$ ;
2 while current_tree  $\neq$  final_tree do
3    $p_{O,free}^G \leftarrow$  current_tree.get_free_target_poses();
4    $O_{free} \leftarrow$  current_tree.get_available_objects();
5    $\mathcal{R}_{free} \leftarrow \{r \in \mathcal{R} : s_{r,current} = \emptyset\}$ ;
6   assignments  $\leftarrow$  assign( $p_{O,free}^G, O_{free}, \mathcal{R}_{free}$ );
7   //
8    $s_{\mathcal{R}} \leftarrow$  create_tasks( $\mathcal{R}_{free}$ , assignments);
9   //
10  any_task_started  $\leftarrow$  False;
11  for  $s_{current,r} \in s_{\mathcal{R}}$  do
12    task_started  $\leftarrow$   $s_{current,r}$ .execute();
13    if task_started then
14      any_task_started  $\leftarrow$  True;
15  //
16  if any_task_started then
17     $t \leftarrow t_{fin,s_{fin},earliest}$ 
18  else
19     $t \leftarrow t_{fin,s_{fin},next}$ 
20  current_tree.synchronize();
21 return  $\emptyset$ ;

```

8.2.2 Planning Phase

The planning phase occurs in lines 3-8 in Algorithm 4. Here, the unassigned poses where targets are placeable (line 3), the assignable objects (line 4), and currently unassigned quadrotors (line 5) are extracted. Following this, the targets are assigned to the target poses and the quadcopters to the objects (line 6). The assignment is done in one of four ways that are presented in later subsections.

Finally, these assignments are translated into quadrotor tasks (line 8), which is explained in the following subsection. The order of occurrence is not obeyed because task mapping creation is essential for understanding problems during assignment.

From Assignments to Tasks

Assignments are tuples of object target poses, objects, and a set of quadrotors required to lift the object: $(p \in p_{O,free}^G, o_n \in O_{free}, \{r_n, \forall n : r_n \in \mathcal{R}_{free}\})$. To translate these to discrete tasks, we use a plan skeleton π . The plan skeleton is the fixed transition sequence that every quadcopter fulfills such that an object reaches its final pose. In the sequence, the drone (1) flies to the object, (2) picks up the object, (3) flies with the object to its designated target position, (4) releases the target, and (5) exits the construction site. Consequently, our plan skeleton is the following (compare [Section 3.2](#)):

$$\begin{aligned} \pi = & [\text{FLY_TO_TARGET}(q_0, \tau_1, q_t), \text{PICK_UP}[o](q_1, p_0, g_2), \\ & \text{FLY_TO_WITH_CARGO}[o](q_1, \tau_3, q_2), \text{RELEASE}[o](q_2, p_4, g_2), \\ & \text{EXIT}(q_5)] \end{aligned}$$

The following list briefly introduces the preconditions, constraints, and effects of the transitions used in this work.

FLY_TO_TARGET (q, τ, q') Fly to an object.

con: $\tau(0) = q, \tau(1) = q', (\forall t \in [0, 1] : \tau(t) \in C_{free})$
pre: $\text{poseQuad} = q$
eff: $\text{poseQuad} \leftarrow q'$

PICK_UP $[o](q, p, g)$ Pick up an object.

con: $\text{Kin}[o](q, p, g), \text{Grasp}[o](g)$
pre: $\text{poseQuad} = q, \text{holding} = \text{None}, \text{pose}[o] = p$
eff: $\text{holding} \leftarrow o, \text{pose}[o] = g$

FLY_TO_WITH_CARGO $[o](q, \tau, q')$ Fly with an object to target position.

con: $\tau(0) = q, \tau(1) = q', (\forall t \in [0, 1] : \tau(t) \in C_{free}), \text{Grasp}[o](g)$
pre: $\text{poseQuad} = q, \text{holding} = o, \text{pose}[o] = g, o_m = o_{m,req}$
eff: $\text{poseQuad} \leftarrow q'$

RELEASE[o](q, p, g) Release a picked object.
con: $\text{Kin}[o](q, p, g), \text{Grasp}[o](g)$
pre: $\text{poseQuad} = q, \text{holding} = o, \text{pose}[o] = g$
eff: $\text{holding} \leftarrow \text{None}, \text{pose}[o] = p$

EXIT(q, τ, q') Leave the construction cite.
con: $\tau(0) = q, \tau(1) = q', (\forall t \in [0, 1] : \tau(t) \in C_{free})$
pre: $\text{poseQuad} = q$
eff: $\text{poseQuad} \leftarrow q'$

In FLY_TO_WITH_CARGO, o_m denotes the current number of attached quadcopters to the object and $o_{m,req}$ the required number. Note that EXIT is required: when the quadcopter stays at its position after RELEASE until it is reassigned, it will block other quadcopters from placing their payload at this pose. Therefore, the exit task was selected to fly to a valid z-position $z = 1$ (x and y are sampled until valid from a Gaussian, with the mean at current x and y values) to prevent blocking others. In the presented definition, it does not differ from FLY_TO_TARGET, whereas in implementation, explicitly defining it as a transition simplifies the process and makes the plan skeleton more explicit.

Using these definitions, for each available quadrotor, a task sequence is created from the known information: initial quadcopter pose q_{init} , current object pose q_{obj} (and therefore desired intermediate quadcopter pose q_1) and the desired final object pose q_{fin} (and therefore desired intermediate quadcopter pose q_2), which is exactly what happens in line 8 of [Algorithm 4](#).

We continue by introducing the assignment algorithms.

The assignment algorithms all have a similar scheme. At first, available objects are assigned to open target poses, and afterward, available quadrotors are assigned to these objects. The assignment of quadrotors to objects is assisted by so-called *attachment points*. These are relative positions on the object where quadrotors can fly to pick up the object. The number of attachment points determines how many quadrotors are required to lift the object and is chosen such that it can not be lifted by fewer.

Random Assignment

The random assignment performs every selection based on a random choice. At first, a random free pose from the set $p_{O,free}^G$ is selected. This pose has a type T (of the expected object), and to this pose, a random object of the same type T is assigned. This is done until either of both sets is empty. Thereafter, another assignment loop starts, where randomly selected samples from the assignments are assigned to quadrotors.

Enabling Coalition Formation At this point, one problem occurs that applies for all of the assignment methods and is relevant to realize coalition formation. A brief example: there are two quadrotors and two objects. Each of them requires two quadrotors to lift. Now the assignment is done, and each of the quadrotors is assigned to one of the objects. At this point, the process will be deadlocked because both quadrotors wait for their partner before the next path can be started, because the precondition $o_m = o_{m,req}$ for FLY_TO_WITH_CARGO (in Figure 8.1) can never be fulfilled.

Therefore, in all assignment methods, there is a pre-selection stage that picks objects until the number of required quadrotors to lift all objects is larger than the number of available quadrotors. This results in one more object than liftable selected, so that all objects can be assigned, and the leftover object can be partly assigned. Note that even though the pre-selection stage does not select all object-to-target-pose assignments, unless otherwise mentioned, all assignments are stored and assignable for quadrotors in the next cycle.

Returning to the random assignment. After the pre-selection stage, which randomly selects a valid amount of objects, the attachment points on the objects are randomly assigned to quadrotors, and eventually, the assignments are returned.

Greedy Intuitive Assignment

The assignment method that was initially and solely used is the "Greedy Intuitive Assignment". It was the first approach to a heuristic and has its (conceptual) flaws. The algorithm prioritizes the objects by the smallest distance of the furthest away quadcopter:

$$prio = \text{sort}(\max(\text{dist}(o, r_i), \forall r_i \in \mathcal{R}_{free})), \forall o \in \mathcal{O}_{free}$$

In the order of appearance these are then assigned to the first found target pose $p \in p_{O,free}^G$ of the same type. To assign quadrotors, the still prioritized assignments are looped, and for each object, the m closest quadrotors are picked, which is the greedy part of the assignment.

Hungarian Assignment

The so far presented heuristics optimized no cost at all or worst-case distance. Since the aim is to find the shortest possible paths for all objects, the problem can be formulated as a classical linear sum assignment problem, see [Section 3.2.1](#). The object-to-target-pose assignment is done with the Hungarian algorithm. Subsequently, of these assignments, a random selection is made for the assignments to the quadrotors. Optimally assigning the intuitive solution, where n quadrotors are assigned to 1 object, is not solvable using the Hungarian. Therefore, not the objects, but their attachment points are used for optimal assignment of quadrotors to objects with the Hungarian.

Greedy Hungarian Assignment with Reassignment

Since the presented basic Hungarian assignment still contains a random component, we looked for a more deterministic solution and came up with the Greedy Hungarian Assignment with Reassignment. Instead of picking randomly, the pre-selection stage greedily selects object-to-target-pose assignments based on the cost of each assignment. Also, this method does not store assignments that were not selected by the pre-selection stage, such that in the next iteration, a better solution can be found if it exists.

8.2.3 Execution Phase

The execution phase is where the task-generated actions are carried out. For each quadrotor, the current action based on the timestep t is selected (line 11). In the next line, 12, the task is attempted to be executed. The success of the process is dependent on the fulfillment of the preconditions and, if these are fulfilled, on the solution availability, see [Section 8.2.5](#). The actions include motion planning (FLY_TO_TARGET, FLY_TO_WITH_TARGET and EXIT) and object manipulation (PICK_UP and RELEASE). To generate the motions, ST-RRT* is utilized as a motion planner. It successfully ensures asymptotically optimal collision-free

multi-robot planning with regard to arrival time. Object manipulation is achieved by attaching and detaching the 3D models of the different bodies to and from each other.

8.2.4 Iteration Preparation Phase

The iteration preparation phase ensures that all tasks can be executed and the construction process does not halt. In [Algorithm 4](#), Lines 10, 13, 14, as well as lines 16-20 can be counted to this phase.

Time Stepping

In normal operation, after all actions are executed, the current timestep t is set to the end time of the earliest finished task from the perspective of the current timestep. This works as long as all tasks succeed and is represented in lines 16-17 of [Algorithm 4](#) (note that the timestep is not allowed to be decreased in line 17). It might happen, however, that the only available task at timestep t cannot be planned at the current timestep (see [Section 8.2.5](#)). In such a case, the end time of the earliest finished task would not change because no new task was started. Therefore, if no task was started (lines 18-19), the current time is increased to the finish time of the next completed action.

After updating the current timestep, the kinematic tree of the environment is updated (line 20) such that new tasks can be assigned or the process terminates if the final tree is a subset of the current tree.

8.2.5 Edge Cases

There is one conceptual edge case that occurs during normal operation that can make a planning step fail. It is best visualized with an example: Imagine a parking lot with two slots that are next to each other, conceptually visualized in [Figure 8.2](#). At time $t = 0$, a car plans a path to the right parking slot that reaches the final position at $t_{atgoal} = 10$. The path crosses the left parking slot but is collision-free. Then, at timestep $t = 1$, a second car plans a path to the left parking slot. Because it is much closer to the slot, it can reach the goal position at $t_{atgoal,2} = 4$. Because the first car is still so far away from the parking slot at $t_{atgoal,2} = 4$, the cars do not collide, and the second path is also collision-free, even though it obstructs the path of the first car.

There are multiple ways to solve this problem. Replanning the first car's path is an intuitive approach, but since we are in the full knowledge instead of the reactive domain, this is not an option. Another approach is backtracking [Har+23], where previous planning steps are rewound and replanned until a solution is found. This is an implementation-intensive but efficient solution since it is only executed when such an edge case occurs. The alternative to backtracking is a specific solution for this goal conflict. The chosen solution is the following: if a path is found for a planning step, the latest end time of a task is taken, and additional collision checking is done for the static end pose of the priorly found path, but for the whole interpolated time between the former paths' end time and the longest paths' endtime. If a collision occurs in this time range, the former path was overtaking another one and is therefore declared invalid. In contrast to backtracking, this puts a larger average load on the process because, for every path, additional collision checking has to be done. Also, just blocking the target zone results in construction time delays, where backtracking could find consecutive solutions. But, as the results have shown in Section 9.3, this time is neglectable.

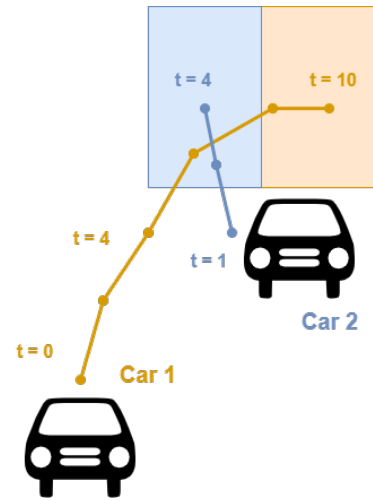


Figure 8.2: Conceptual Visualization of the Takeover Case with (holonomic) Cars

The following chapter contains the experimental section. It begins by introducing the three created rearrangement scenarios, and continues by showing the methodology, the experiment measures, the results, and concludes with a discussion of the outcomes.

9.1 Scenarios

We present three different scenarios that have been chosen for demonstration of different measures and evaluation of the used assignment algorithms.

9.1.1 Hollow Jenga Refactored Scenario

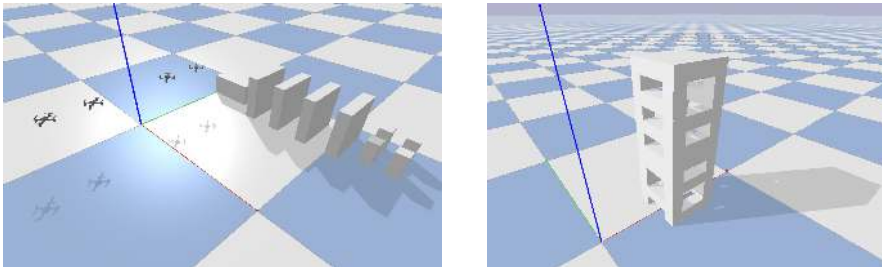


Figure 9.1: Initial and final state of the hollow jenga refactored scenario

"Hollow Jenga Refactored" originates from the Jenga tower, the basis for revision 1 of this scenario, where a tower of layers of three elongated cuboids, varying in angle, is built. In the presented version, the middle cuboids are left out, and several layers are customized by exchanging the blocks with cubes or a plane that occupies the whole layer. The scenario serves the purpose of presenting different building block capabilities in a clear and concise way.

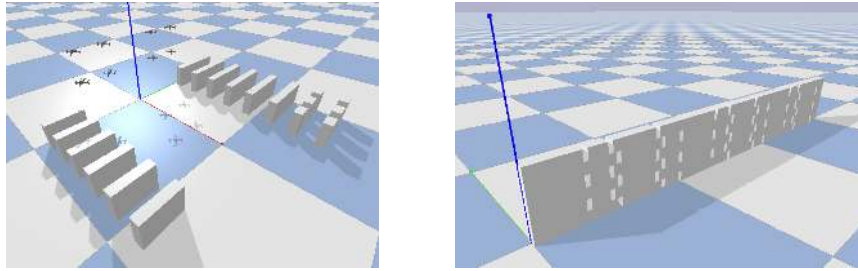


Figure 9.2: Initial and final state of the wall scenario

9.1.2 Wall Scenario

The wall scenario provides a way to test the scalability properties of the approach. Here, many object positions and placeable objects are simultaneously available throughout the entire run.

9.1.3 House Scenario

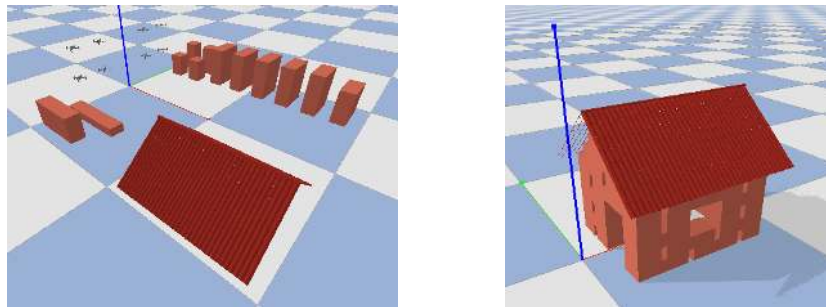


Figure 9.3: Initial and final state of the house scenario

The house scenario tries to simulate a somewhat more realistic environment that combines properties of both previous approaches in a constrained way, where a variety of different blocks is used with a varying (sometimes strongly limited) amount of open positions for potential objects.

9.1.4 Dimensions

The following table [Table 9.1.4](#) contains the scenario dimensions, described by the number of objects of each type. We differentiate between the number of building blocks and the number of required agents to lift these because the effort to fulfill a scenario of 10 blocks that require three agents to lift versus a scenario of 10 blocks that each requires one agent to lift is three times as much.

Scenario	n_{o-1}	n_{o-2}	n_{o-3}	n_{o-4}	n_{o-6}	n_{total}	n_{lifts}
Hollow Jenga	8	16	-	2	-	26	48
Wall	18	18	18	-	-	54	108
House	16	16	3	1	1	37	67

Table 9.1: Amount of objects for all scenarios, differentiated by number of agents required to lift

9.2 Methodology

Because there are no benchmarks of similar approaches, the random assignment serves as a virtual baseline to compare the different assignment variants. Before presenting the experiment data, a task map for each scenario is provided. In the first experiment, in [Section 9.3.2](#), we compare the different assignment method performances for each scenario, each with eight agents. The measure is the total construction time from the initial to the final state. The comparison is done with violin plots, where the middle blue line represents the mean, the grey line represents the median, and the light blue area represents the distribution, with the x-expansion being relative. The second experiment, in [Section 9.3.3](#), tests the scalability of the complete algorithm¹ and construction time with an increasing number of robots using only a single assignment approach, namely the Hungarian.

9.3 Results and Discussion

We begin by presenting a visualization of the task space over the quadcopters to show the task dimensionality of each scenario and potential complexity of

¹ The tests are run in Ubuntu 20.04, on an Intel Core i5-10210U CPU @ 1.60Ghz with 16GB RAM

the process in Figures 9.4, 9.5 and 9.6. Here, the task assignments of eight quadcopters over the whole construction time of are presented.

9.3.1 Task Map

Due to their short execution times, the pick-up and release tasks are barely visible. The "WAITING" state is active when a quadrotor is attached to its object-to-lift, but not all partners to lift the object are present yet. Quadrotors are "IDLE", if no task was assigned to them.

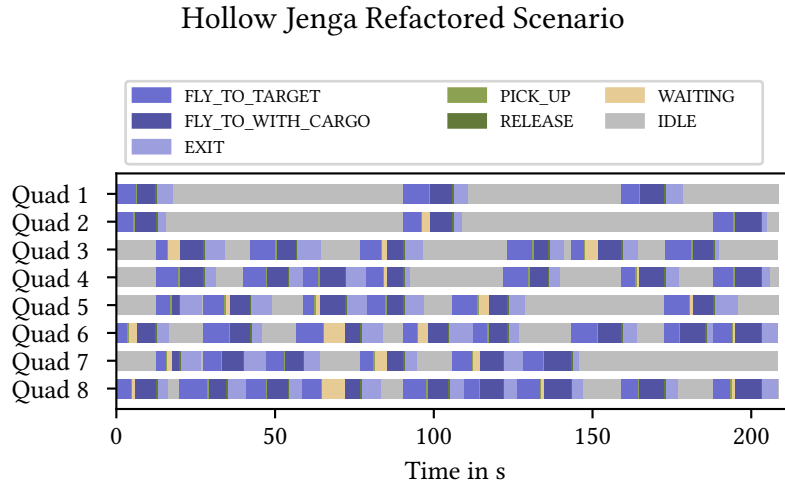


Figure 9.4: Task assignments of 8 quadcopters for the hollow jenga refactored scenario planned with Hungarian assignment.

Discussion

The first figure, Figure 9.4 is a taskmap of the most compact hollow jenga refactored scenario. A great percentage of it contains idle times, four quadcopters are not being assigned at the beginning $t = 0$. The reason for this is the limited availability of placeable objects, that is two or max three at the same time. The scenario requires at least four quadcopters because a 4-object has to be lifted, but already with four robots the scenario is nearly saturated.

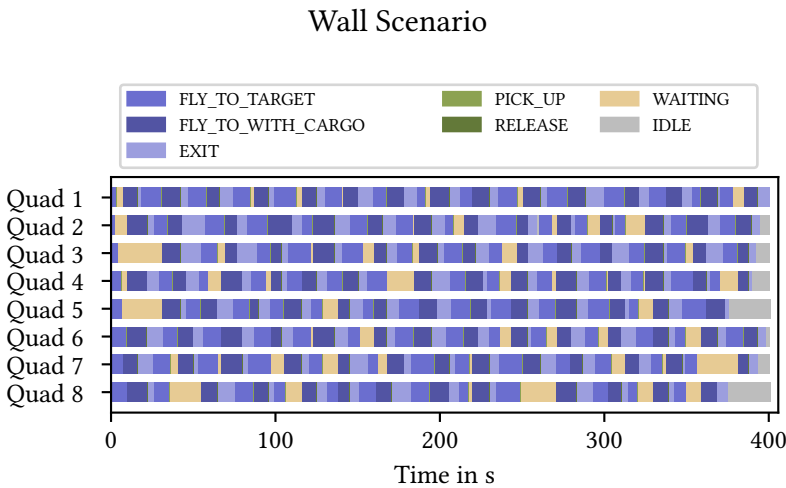


Figure 9.5: Task assignments of 8 quadcopters for the wall scenario planned with Hungarian assignment.

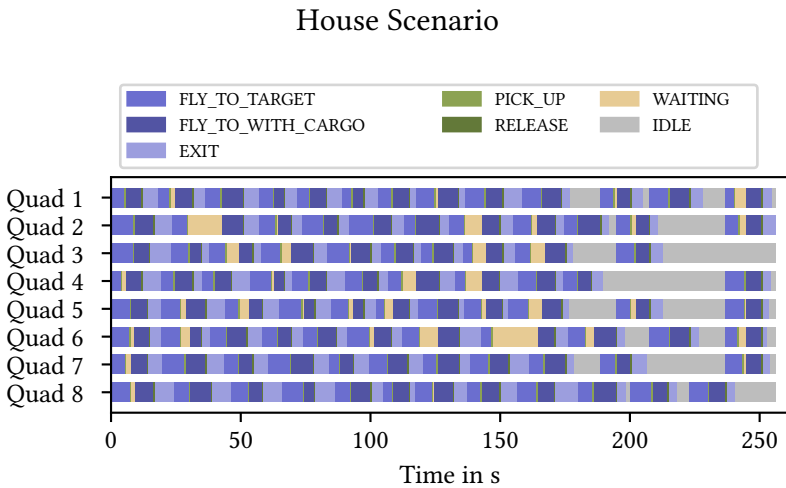


Figure 9.6: Task assignments of 8 quadcopters for the house scenario planned with Hungarian assignment.

The taskmap of the wall scenario, Figure 9.5, contains some representative examples for "WAITING" situations. Right at the beginning, quadcopters 3 and 5 have planned to a 3-object they want to lift and are waiting until quadcopter 1 has finished placing its first object, exiting and flying to the common object.

Clearly noticeable are the idle times at the end of the house scenario in Figure 9.6. These are characteristic of the house scenario and stem from the quadcopters waiting for all the bricks the roof depends on to be placed. Thereafter, the roof is placed by hiring six robots.

9.3.2 Assignment Algorithm Performance

Figures 9.7, 9.8 and 9.9 show the construction time distributions of 100 runs of each scenario. For better readability, the Greedy Intuitive assignment is abbreviated GREEDY_I, and the base Hungarian and Hungarian Greedy Switching assignments are abbreviated HUNG and HUNG_GS, respectively.

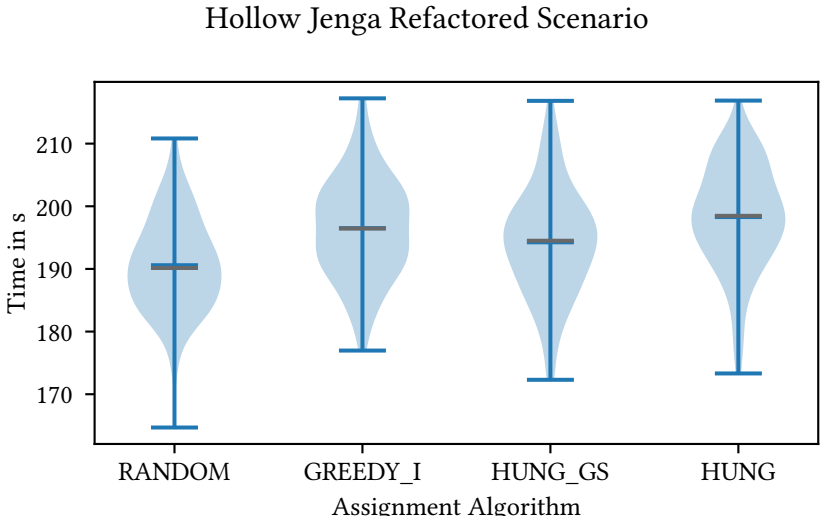


Figure 9.7: Construction time distribution for the hollow jenga refactored scenario with 8 quadrotors over 100 runs each.

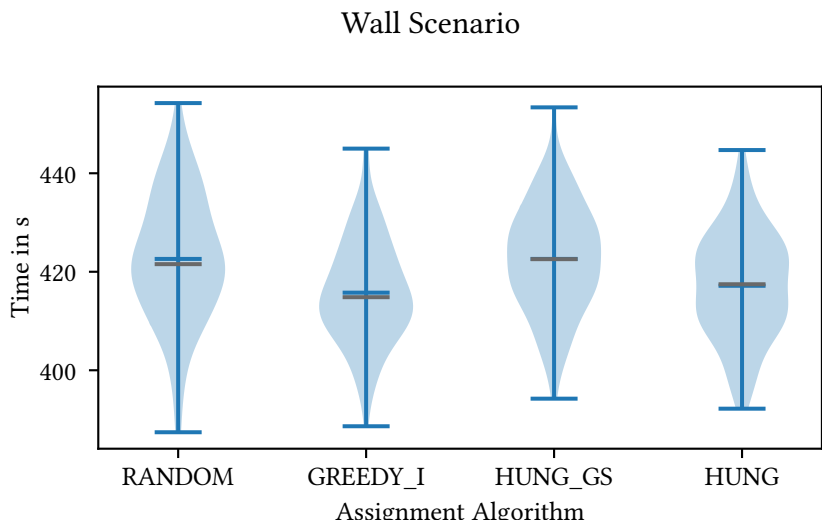


Figure 9.8: Construction time distribution for the wall scenario with 8 quadrotors over 100 runs each.

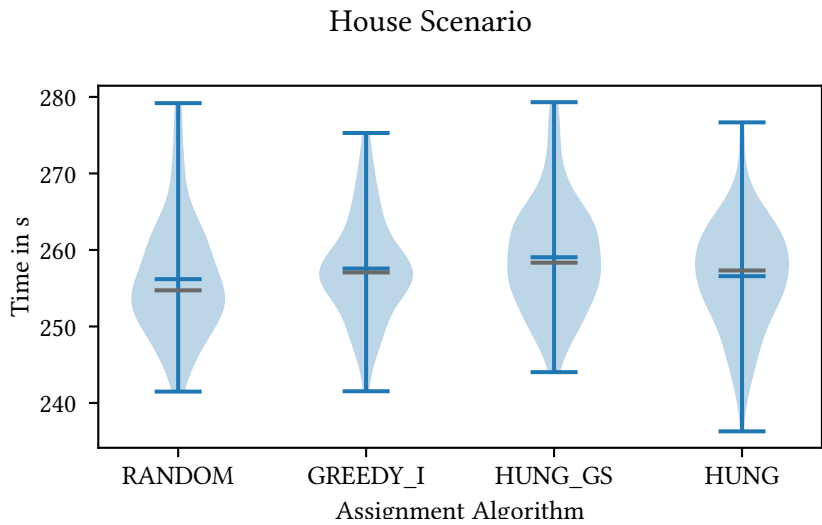


Figure 9.9: Construction time distribution for the house scenario with 8 quadrotors over 100 runs each.

Discussion

While the extrema of each algorithm differ by some minor seconds, the means and medians are also neglectably close to each other. Independent of the scenario, the tendency is similar. Even though the Hungarian variants in the wall scenario in Figure 9.8 are symmetrical and other systematics can be interpreted, these systematics are scenario-dependent, and the distributions differ from scenario to scenario. In the hollow jenga refactored scenario, Figure 9.7, even the random assignment is the fastest in the room. But still, by a neglectable factor. The Hungarian method is an optimal assignment algorithm. But, since it only finds the optimal solution for snapshots and not for the global assignments, it, by far, does not provide a globally optimal solution. So, optimality loses its effect when, later in construction, objects have to be gathered from "outer space". Moreover, there is an optimal total linear sum $\sum^{opt}$ of all object $\rightarrow$ object target position assignments and a worst-case total linear sum $\sum^{worst-case}$ that limit the potential distance from the bottom and the top, $t_{construction} \in [\sum^{opt}, \sum^{worst-case}]$. Because an anti-greedy heuristic that always picks the worst possible assignment must also choose the "short" paths, it cannot select infinitely long paths and, therefore, arbitrarily worsen the solution. In our cases, these boundaries are not far apart, as approximately demonstrated by the random heuristic, and locally optimal assignment can not find globally optimal solutions. When the assignment of quadrotors is considered, these absolute borders increase but do not change the concept. Since ST-RRT*, constrained by v_{max} of the robot, asymptotically finds optimal solutions with respect to the shortest arrival time, this can also be applied to the time dimension. In conclusion, differentiating between heuristics might lead to small tendencies regarding performance, but over all scenarios, these tendencies are neglectable, and choosing a certain heuristic will not bring notable improvements. This might differ if a global assignment approach is applied.

9.3.3 Scalability

To test the scalability, the scenarios will be executed using the Hungarian heuristic with different numbers of agents for 30 runs each. The result is shown in figures 9.10, 9.11 and 9.12 together with the planning time distribution over each step. In the plots, the thick line in the middle is the mean, whereas the thinner lines are the 75- and 25-percentile. Because the percentiles are non-centered

medians and the mean is not the median, the mean can be "pulled out" of the shaded area by a small imbalance of a few large extreme values. For better comparability, the y-axis is fixed from 0 to 1200s.

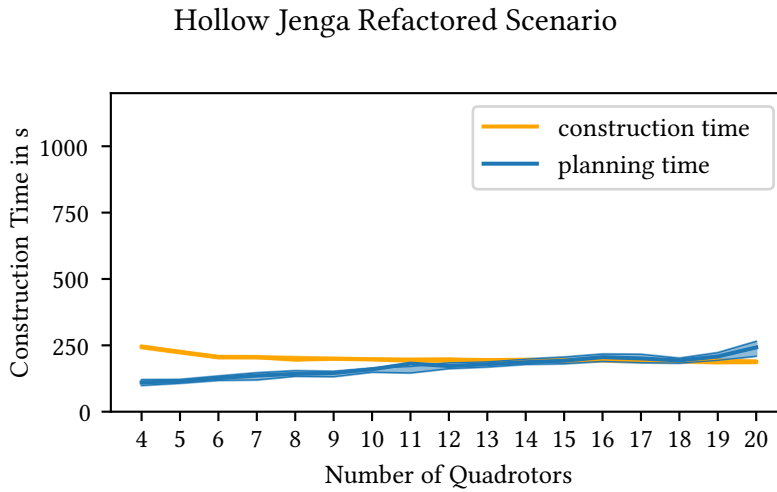


Figure 9.10: Hollow Jenga Refactored scenario construction and planning time, dependent on the number of quadrotors.

Results

The first plot, of the Hollow Jenga Refactored scenario, [Figure 9.10](#) shows that the construction time starts slightly above 200 seconds with four quadrotors and is slightly below 200 seconds for 20 quadrotors. It is, therefore, approximately constant. The planning time ranges from 100s to slightly over 200s. In [Figure 9.11](#) of the wall scenario, with the increasing number of agents, the construction time shrinks by a factor of 6 from around 1200s to under 200s. This decline takes place in the shape of an $e^{\frac{1}{x}}$ function. At around 20 quadrotors, the slope of the function approximates 0. At the same time, the planning time increases approximately linearly from around 200s to a value between about 500s and 600s. In the final [Figure 9.12](#) of the house scenario, the planning time ranges with an approximately linear slope from about 250s at 6 quadrotors to 600s at 20 quadrotors, while the construction time decreases from around 350s to 200s.

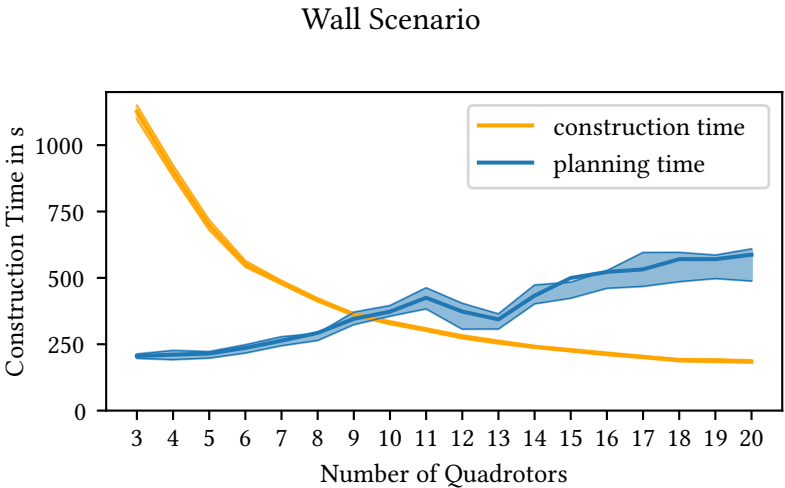


Figure 9.11: Wall scenario construction and planning time, dependent on the number of quadrotors.

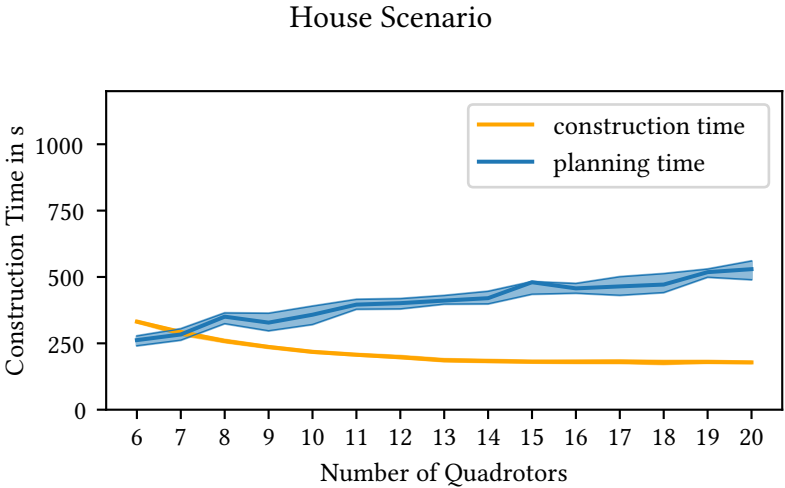


Figure 9.12: House scenario construction and planning time, dependent on the number of quadrotors.

Discussion

The construction time can be much more influenced by a different factor: the number of assembly robots through parallelization, with the cost of increasing planning times due to additional collision checking. This is the hypothesis for the upcoming discussion. [Figure 9.11](#) confirms that the hypothesis is correct. With an increasing number of quadrotors, the construction time can be reduced by a factor of 6 until, at 20 quadrotors, the decline stagnates due to saturation. This saturation is also clearly visible in [Figure 9.13](#), where right from the start,

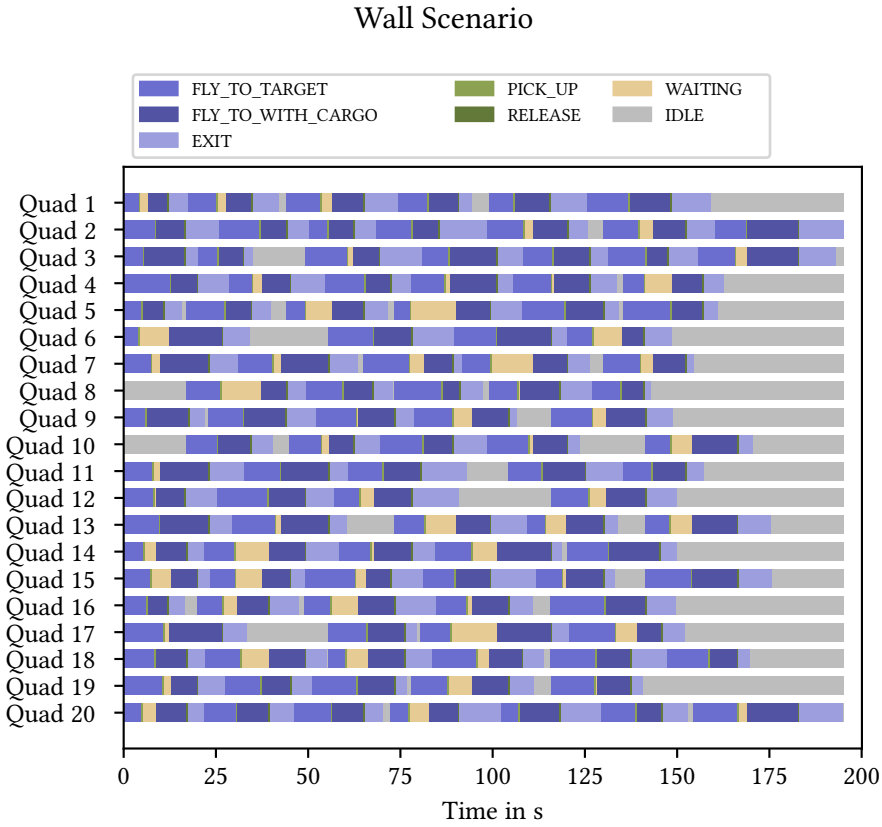


Figure 9.13: Example run for the wall scenario with 20 quadrotors, planned with Hungarian Assignment

two quadcopters stay unassigned and similar to the hollow jenga task assignment in [Figure 9.4](#). The only speed-up factor they contribute is their instant availability after the first blocks are placed, while the other robots are exiting after releasing their payload. This behavior is visible throughout the whole process, where, sporadically, nearly every quadcopter has idle times. Parallel to the shrinking construction time, the planning time increases approximately linearly, as expected. This is the case because, for more parallelly flying quadcopters, more collision checking has to be done, which is commonly the most costly operation in motion planning. Note that the presented graph is by far not linear in its first half and not representative of a real single-threaded run. The reason for these inconsistencies is that multiple threads with varying loads and other parallel running processes interfered with the runs. Running solely one process simultaneously is expected to have fewer outliers, like the curve starting at around 8 quadrotors.

The other results, [Figure 9.10](#) and [Figure 9.12](#), seem to disagree with the hypothesis but do not and rather show a critical point. The construction time does not arbitrarily scale with the number of applied quadrotors, but the scalability is solely dependent on the parallelization capabilities of the scenario. The wall scenario was explicitly designed to constantly have a high number of available object target poses and targets, which does not hold for the others. Hence, the hollow jenga scenario that has at max three open target poses at the same time can only improve its construction time by a minimal amount with the increasing number of quadrotors. Still, the planning time increases approximately linearly because for each additional quadrotor, the collision checking effort increases. The results of these experiments also support the result from [Section 9.3.2](#). We see that the variance of the orange curve is very small in contrast to the changes that a single additional robot can make to the construction time. For the Hungarian assignment that is presented here, this result also holds throughout the whole spectrum of quadrotor numbers.

In a real-world scenario, the number of available quadcopters might be limited by money, availability, or other constraining quantities. Therefore, it can be useful to minimize the number of quadrotors with regard to the planning and construction times. This information is already contained in the figures [9.11](#) – [9.12](#) and can be explicitly extracted by multiplication of the normalized measures, the construction, and the planning times, see [Figure 9.15](#). Minimizing

the resulting function gives the number of quadrotors with the lowest planning and construction time cost. From figures 9.14, 9.15 and 9.16 can be read that this cost is minimal at 5 quadrotors for the hollow jenga scenario, 9 for the house scenario and 13 for the wall scenario. For the wall scenario, this is the case because of the dip in planning time that probably exists due to too few and ambiguous runs. Because in the higher numbers of quadrotors, the product of the functions is very low due to low construction time, it might be necessary to weight this function in favor of the planning time to get better results.

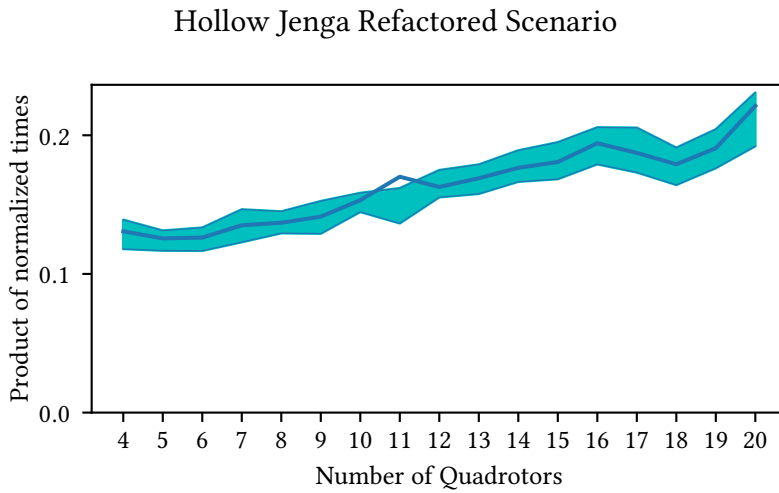


Figure 9.14: Products of normalized construction and simulation times with mean, 75- and 25-percentiles. Minimum at 13 Quadrotors.

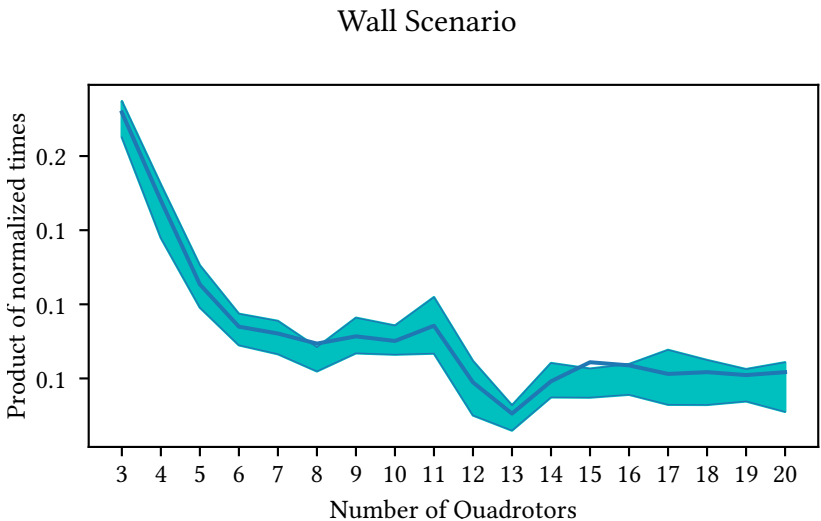


Figure 9.15: Products of normalized construction and simulation times with mean, 75- and 25-percentiles

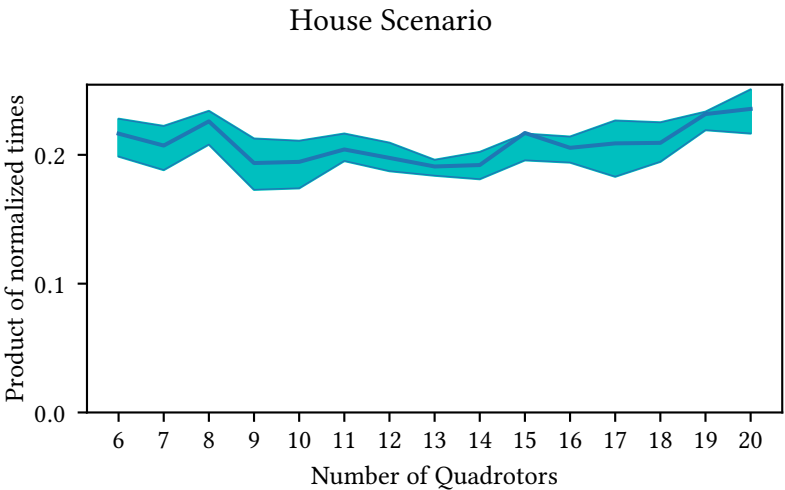


Figure 9.16: Products of normalized construction and simulation times with mean, 75- and 25-percentiles

We have shown that the long-horizon multi-agent rearrangement problem is solvable for quadrotors using a task and motion planning approach that identifies subproblems and sequentially solves these. The work of [Har+23] was extended by coalition formation such that quadrotors can form teams to lift heavy objects in assembly tasks cooperatively.

This result was achieved by the implementation of a Python framework that incorporates the capabilities of the Open Motion Planning Library for motion planning, namely the planning algorithm ST-RRT* and the physics simulator pyBullet for collision checking and visualization. Four assignment methods for the selection of subproblems were introduced and thoroughly tested in three scenarios using this framework. Additionally, the scalability of the algorithm and the construction time were evaluated, both with regard to an increasing number of quadrotors.

For the first experiments, we found that considering the total construction time, the choice of the assignment method does not influence the result in a relevant manner. All four algorithms stayed in similar construction time bounds, and nearly each of the methods was "the fastest" in one of the scenarios, but always with a neglectable lead.

For scalability, the result was more differentiable. We showed that depending on the scenario, the construction time could be reduced by a factor of up to 6 when increasing the number of quadrotors, while the planning time increased approximately linearly by a factor of 3. Both construction time and planning time heavily depend on the scenario. While the construction time is a scalability measure of the scenario only, the planning time scales with the number of agents as additional efforts for collision checking are required.

This is not the case for the scalability of the algorithms' planning time. Since for every additional agent, the collision checking effort increases, it is not solely dependent on the scenario but on the number of agents. Still, it is influenced by the scenario properties because, in saturation, few quadcopters are planned for, and therefore, collision-checking is cheap. Consequently, the approximately

linear increase of computational effort is realistic when not in saturation. When the scenario is saturated with enough quadcopters, it is rather an upper bound than the expected real value.

A more systematic and predictable approach to improve the construction time with heuristics is applying the heuristics from a global perspective and globally finding a feasible and optimal assignment order. While this approach could improve performance, this is not guaranteed because it does not respect the unknown time horizon of ST-RRT*. Also, as shown in [Section 9.3.2](#), the total construction time is lower bound, and therefore, investigating this method is not likely to increase the construction performance by a notable degree.

A more promising approach to decrease the construction time is analyzing the planned paths. Currently, approximately only a third of planned paths are actually "value generating", which is when objects are transported. The other paths consist of flying to an object and leaving the construction site. Especially the exit planner, which finds a valid pose at $z = 1$ and goes there, has lots of potential for improvement. One idea is to introduce a "construction area" and a "stock area". When, after a successful payload release, a quadcopter leaves in the direction of where the building blocks are stored, the overhead way that is generated by exiting to "somewhere above the construction site" and then replanning to the next object can be minimized. A second combinable approach is an adaptive z-value exit planner. Here, a valid exit z-height can be adaptively determined by the currently highest building block and the highest currently placeable object.

For real-world applications, it might be useful to combine the construction capabilities of quadrotors with the ones of static or ground-bound robots. These can complement each other to assemble buildings efficiently. For this, several more challenges have to be overcome: in this work, we assume full world-state knowledge. Direct simulation information is not comparable to the uncertain data of real-world online SLAM. Also, the presented method is too slow for online replanning, and no grasp dynamics are active. To be able to grasp objects, a feasible attachment mechanism has to be found. A gripper can be used, but a capable (6-DOF) variant increases the state-space dimensionality of the quadcopter, which, in real-world scenarios, is often already 12 or 18-dimensional. The additional degrees of freedom lead to an exponential growth of the state space and, therefore, increased planning effort.

Still, solutions for such challenges are constantly developed, and it is conceivable and exciting that teams of quadrotors can lift the largest existing industry to new levels of efficiency.

Bibliography

- [BDM09] Rainer E. Burkard, Mauro Dell’Amico, and Silvano Martello. **Assignment Problems**. SIAM, 2009. ISBN: 978-0-89871-663-4. DOI: [10.1137/1.9781611972238](https://doi.org/10.1137/1.9781611972238). URL: <https://doi.org/10.1137/1.9781611972238> (see page 26).
- [BDT17] M. A. Boon, A. P. Drijfhout, and S. Tesfamichael. **Comparison of a fixed-wing and multi-rotor UAV for Environmental Mapping Applications: A Case Study**. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences XLII-2-W6* (Aug. 2017). Conference Name: International Conference on Unmanned Aerial Vehicles in Geomatics (Volume XLII-2/W6) - 7 September 2017, Bonn, Germany Publisher: Copernicus GmbH, 47–54. ISSN: 1682-1750. DOI: [10.5194/isprs-archives-XLII-2-W6-47-2017](https://isprs-archives-XLII-2-W6-47-2017). URL: <https://isprs-archives.copernicus.org/articles/XLII-2-W6/47/2017/isprs-archives-XLII-2-W6-47-2017.html> (see page 2).
- [Bin+20] Xue Bin Peng, Erwin Coumans, Tingnan Zhang, Tsang-Wei Lee, Jie Tan, and Sergey Levine. **Learning Agile Robotic Locomotion Skills by Imitating Animals**. In: *Robotics: Science and Systems XVI*. Robotics: Science and Systems Foundation, July 2020. ISBN: 978-0-9923747-6-1. DOI: [10.15607/RSS.2020.XVI.064](https://doi.org/10.15607/RSS.2020.XVI.064). URL: <http://www.roboticsproceedings.org/rss16/p064.pdf> (see page 41).
- [Bit23a] Bitcraze. *Controllers in the Crazyflie*. July 2023. URL: <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/> (visited on 10/21/2023) (see page 30).
- [Bit23b] Bitcraze. *Crazyflie Firmware*. Oct. 2023. URL: <https://github.com/bitcraze/crazyflie-firmware> (visited on 10/21/2023) (see page 31).
- [Bit23c] Bitcraze. *PWM to Thrust*. July 2023. URL: <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/pwm-to-thrust/> (visited on 10/21/2023) (see page 31).
- [BK00] R. Bohlin and L.E. Kavraki. **Path planning using lazy PRM**. In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. San Francisco, CA, USA: IEEE, 2000, 521–528 vol.1. ISBN: 978-0-7803-5886-7. DOI: [10.1109/ROBOT.2000.844107](https://doi.org/10.1109/ROBOT.2000.844107). URL: <http://ieeexplore.ieee.org/document/844107/> (see pages 20, 21).

- [BOV99] V. Boor, M.H. Overmars, and A.F. Van Der Stappen. **The Gaussian sampling strategy for probabilistic roadmap planners**. In: *Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No. 99CH36288C)*. Vol. 2. Detroit, MI, USA: IEEE, 1999, 1018–1023. ISBN: 978-0-7803-5180-6. DOI: [10.1109/ROBOT.1999.772447](https://doi.org/10.1109/ROBOT.1999.772447). URL: <http://ieeexplore.ieee.org/document/772447/> (see page 20).
- [Boy17] Michael Boyle. **The Integration of Angular Velocity**. *Advances in Applied Clifford Algebras* 27:3 (Sept. 2017), 2345–2374. ISSN: 0188-7009, 1661-4909. DOI: [10.1007/s00006-017-0793-z](https://doi.org/10.1007/s00006-017-0793-z). URL: <http://link.springer.com/10.1007/s00006-017-0793-z> (see page 28).
- [BWM17] Filipe Barbosa, Jonathan Woetzel, and Jan Mischke. **Reinventing construction: A route of higher productivity**. Tech. rep. McKinsey Global Institute, 2017 (see page 1).
- [CB16] Erwin Coumans and Yunfei Bai. *PyBullet, a Python module for physics simulation for games, robotics and machine learning*. 2016. URL: <http://pybullet.org> (visited on 10/15/2023) (see page 41).
- [Cdb22] Penrose Cdb. **2022 Global Status Report for Buildings and Construction** (2022) (see page 1).
- [Cho23] Alka Choudhary. *Sampling-based Path Planning Algorithms: A Survey*. arXiv:2304.14839 [cs]. Apr. 2023. URL: <http://arxiv.org/abs/2304.14839> (see page 21).
- [Col+21] Jack Collins, Shelvin Chand, Anthony Vanderkop, and David Howard. **A Review of Physics Simulators for Robotic Applications**. *IEEE Access* 9 (2021), 51416–51431. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2021.3068769](https://doi.org/10.1109/ACCESS.2021.3068769). URL: <https://ieeexplore.ieee.org/document/9386154/> (see pages 41, 42).
- [Eri+19] Zackory Erickson, Vamsee Gangaram, Ariel Kapusta, C. Karen Liu, and Charles C. Kemp. *Assistive Gym: A Physics Simulation Framework for Assistive Robotics*. arXiv:1910.04700 [cs]. Oct. 2019. URL: <http://arxiv.org/abs/1910.04700> (see page 41).
- [FHL08] Dave Ferguson, Thomas M. Howard, and Maxim Likhachev. **Motion planning in urban environments**. *Journal of Field Robotics* 25:11-12 (Nov. 2008), 939–960. ISSN: 1556-4959, 1556-4967. DOI: [10.1002/rob.20265](https://doi.org/10.1002/rob.20265). URL: <https://onlinelibrary.wiley.com/doi/10.1002/rob.20265> (see pages 10, 11).
- [FN13] Emil Fresk and George Nikolakopoulos. **Full quaternion based attitude control for a quadrotor**. In: *2013 European Control Conference (ECC)*. Zurich: IEEE, July 2013, 3864–3869. ISBN: 978-3-033-03962-9. DOI: [10.23919/ECC.2013.6669617](https://doi.org/10.23919/ECC.2013.6669617). URL: <https://ieeexplore.ieee.org/document/6669617/> (see page 29).

- [Gar+20] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. *Integrated Task and Motion Planning*. arXiv:2010.01083 [cs]. Oct. 2020. URL: <http://arxiv.org/abs/2010.01083> (see pages 23–25).
- [Gro+22] Francesco Grothe, Valentin N. Hartmann, Andreas Orthey, and Marc Toussaint. *ST-RRT*: Asymptotically-Optimal Bidirectional Motion Planning through Space-Time*. arXiv:2203.02176 [cs]. Mar. 2022. URL: <http://arxiv.org/abs/2203.02176> (see pages 35, 43).
- [Hal20] Dan Halperlin. *Lecture notes in Algorithmic Robotics and Motion Planning*. Sept. 2020 (see page 17).
- [Har+23] Valentin Noah Hartmann, Andreas Orthey, Danny Driess, Ozgur S. Oguz, and Marc Toussaint. **Long-Horizon Multi-Robot Rearrangement Planning for Construction Assembly**. *IEEE Transactions on Robotics* 39:1 (Feb. 2023). arXiv:2106.02489 [cs], 239–252. ISSN: 1552-3098, 1941-0468. DOI: [10.1109/TRO.2022.3198020](https://doi.org/10.1109/TRO.2022.3198020). URL: <http://arxiv.org/abs/2106.02489> (see pages 1, 2, 35, 36, 39, 47, 56, 71).
- [HLW00] D. Halperin, J.-C. Latombe, and R. H. Wilson. **A General Framework for Assembly Planning: The Motion Space Approach**. *Algorithmica* 26:3-4 (Mar. 2000), 577–601. ISSN: 0178-4617, 1432-0541. DOI: [10.1007/s004539910025](https://doi.org/10.1007/s004539910025). URL: <http://link.springer.com/10.1007/s004539910025> (see page 24).
- [HO22a] Wolfgang Hönig and Andreas Orthey. *Lecture notes in Motion Planning - Lecture 1 - Organization, Introduction, Problem Formulation*. Apr. 2022 (see pages 7, 8).
- [HO22b] Wolfgang Hönig and Andreas Orthey. *Lecture notes in Motion Planning - Lecture 12 - Search-, Sampling-, and Optimization-based Motion Planning*. July 2022 (see page 22).
- [HO22c] Wolfgang Hönig and Andreas Orthey. *Lecture notes in Motion Planning - Lecture 2 - Topology, Metrics, Collision Checking*. Apr. 2022 (see page 8).
- [HO22d] Wolfgang Hönig and Andreas Orthey. *Lecture notes in Motion Planning - Lecture 4 - Advanced Search-Based Motion Planning*. May 2022 (see page 9).
- [Hsu+03] D. Hsu, Tingting Jiang, J. Reif, and Zheng Sun. **The bridge test for sampling narrow passages with probabilistic roadmap planners**. In: *2003 IEEE International Conference on Robotics and Automation (Cat. No.03CH37422)*. Vol. 3. ISSN: 1050-4729. Sept. 2003, 4420–4426 vol.3. DOI: [10.1109/ROBOT.2003.1242285](https://doi.org/10.1109/ROBOT.2003.1242285) (see page 20).

- [Kal+11] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. **STOMP: Stochastic trajectory optimization for motion planning**. In: *2011 IEEE International Conference on Robotics and Automation*. ISSN: 1050-4729. May 2011, 4569–4574. DOI: [10.1109/ICRA.2011.5980280](https://doi.org/10.1109/ICRA.2011.5980280) (see page 11).
- [Kel+06] Alonzo Kelly, Anthony Stentz, Omead Amidi, Mike Bode, David Bradley, Antonio Diaz-Calderon, Mike Hoppold, Herman Herman, Robert Mandelbaum, Tom Pilarski, Pete Rander, Scott Thayer, Nick Vallidis, and Randy Warner. **Toward Reliable Off Road Autonomous Vehicles Operating in Challenging Environments**. *The International Journal of Robotics Research* 25:5-6 (May 2006), 449–483. ISSN: 0278-3649, 1741-3176. DOI: [10.1177/0278364906065543](https://doi.org/10.1177/0278364906065543). URL: <http://journals.sagepub.com/doi/10.1177/0278364906065543> (see page 10).
- [KF11] Sertac Karaman and Emilio Frazzoli. *Sampling-based Algorithms for Optimal Motion Planning*. arXiv:1105.1186 [cs]. May 2011. URL: <http://arxiv.org/abs/1105.1186> (see pages 16–18, 21).
- [KH04] N. Koenig and A. Howard. **Design and use paradigms for gazebo, an open-source multi-robot simulator**. In: *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat.No.04CH37566)*. Vol. 3. Sendai, Japan: IEEE, 2004, 2149–2154. ISBN: 978-0-7803-8463-7. DOI: [10.1109/IROS.2004.1389727](https://doi.org/10.1109/IROS.2004.1389727). URL: <http://ieeexplore.ieee.org/document/1389727/> (see page 42).
- [KI08] E. Koyuncu and G. Inalhan. **A probabilistic B-spline motion planning algorithm for unmanned helicopters flying in dense 3D environments**. In: *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Nice: IEEE, Sept. 2008, 815–821. ISBN: 978-1-4244-2057-5 978-1-4244-2058-2. DOI: [10.1109/IROS.2008.4651122](https://doi.org/10.1109/IROS.2008.4651122). URL: <http://ieeexplore.ieee.org/document/4651122/> (see page 19).
- [KL00] J.J. Kuffner and S.M. LaValle. **RRT-connect: An efficient approach to single-query path planning**. In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. Vol. 2. San Francisco, CA, USA: IEEE, 2000, 995–1001. ISBN: 978-0-7803-5886-7. DOI: [10.1109/ROBOT.2000.844730](https://doi.org/10.1109/ROBOT.2000.844730). URL: <http://ieeexplore.ieee.org/document/844730/> (see pages 15, 18, 19).
- [LaV06] Steven Michael LaValle. **Planning algorithms**. OCLC: ocm65301992. Cambridge ; New York: Cambridge University Press, 2006. ISBN: 978-0-521-86205-9 (see pages 11, 14).

- [Liu+17] Sikang Liu, Nikolay Atanasov, Kartik Mohta, and Vijay Kumar. **Search-based motion planning for quadrotors using linear quadratic minimum time control**. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. ISSN: 2153-0866. Sept. 2017, 2872–2879. DOI: [10.1109/IROS.2017.8206119](https://doi.org/10.1109/IROS.2017.8206119) (see page 10).
- [LN16] Carlos Luis and Jérôme Le Ny. *Design of a Trajectory Tracking Controller for a Nanoquadcopter*. arXiv:1608.05786 [cs]. Aug. 2016. URL: <http://arxiv.org/abs/1608.05786> (see pages 28, 30).
- [Mal+21] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, Thomas Lew, Riccardo Bonalli, Marco Pavone, and Behcet Acikmese. *Convex Optimization for Trajectory Generation*. arXiv:2106.09125 [cs, eess, math]. June 2021. URL: <http://arxiv.org/abs/2106.09125> (see pages 11–13).
- [Mic04] O. Michel. **Webots: Professional Mobile Robot Simulation**. *Journal of Advanced Robotics Systems* 1:1 (2004), 39–42. URL: <http://www.ars-journal.com/International-Journal-of-Advanced-Robotic-Systems/Volume-1/39-42.pdf> (see page 42).
- [Nät+17] Mathias Näther, Venkatesh Naidu Nerella, Martin Krause, Günter Kunze, Viktor Mechtcherine, and Rainer Schach. **Beton-3D-Druck - Machbarkeit-untersuchungen zu kontinuierlichen und schalungsfreien Bauverfahren durch 3D-Formung von Frischbeton: Abschlussbericht**. German. Forschungsinitiative Zukunft Bau F 3024. Stuttgart: Fraunhofer IRB Verlag, 2017. ISBN: 978-3-7388-0028-9 (see page 1).
- [Ort] Andreas Orthey. *Configuration Space Visualizer*. URL: <https://aorthy.github.io/configuration-space-visualizer/js-cspace/> (visited on 10/31/2023) (see page 8).
- [Pan+21] Jacopo Panerati, Hehui Zheng, SiQi Zhou, James Xu, Amanda Prorok, and Angela P. Schoellig. *Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control*. Issue: 1 Pages: 1–8 Publication Title: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) Volume: 1. Jan. 2021. URL: <https://github.com/utiasDSL/gym-pybullet-drones> (see page 42).
- [PK05] M. Pivtoraiko and A. Kelly. **Generating near minimal spanning control sets for constrained motion planning in discrete state spaces**. In: *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*. ISSN: 2153-0866. Aug. 2005, 3231–3237. DOI: [10.1109/IROS.2005.1545046](https://doi.org/10.1109/IROS.2005.1545046) (see pages 9, 10).
- [Qua17] Quan Quan. **Introduction to Multicopter Design and Control**. 1st ed. 2017. Singapore: Springer Singapore : Imprint: Springer, 2017. ISBN: 978-981-10-3382-7. DOI: [10.1007/978-981-10-3382-7](https://doi.org/10.1007/978-981-10-3382-7) (see page 28).

- [Rat+09] Nathan Ratliff, Matt Zucker, J. Andrew Bagnell, and Siddhartha Srinivasa. **CHOMP: Gradient optimization techniques for efficient motion planning**. In: *2009 IEEE International Conference on Robotics and Automation*. Kobe: IEEE, May 2009, 489–494. ISBN: 978-1-4244-2788-8. DOI: [10.1109/ROBOT.2009.5152817](https://doi.org/10.1109/ROBOT.2009.5152817). URL: <http://ieeexplore.ieee.org/document/5152817/> (see pages 11, 12).
- [RSF13] Eric Rohmer, Surya P. N. Singh, and Marc Freese. **V-REP: A versatile and scalable robot simulation framework**. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Tokyo: IEEE, Nov. 2013, 1321–1326. ISBN: 978-1-4673-6358-7 978-1-4673-6357-0. DOI: [10.1109/IROS.2013.6696520](https://doi.org/10.1109/IROS.2013.6696520). URL: <http://ieeexplore.ieee.org/document/6696520/> (see page 42).
- [Sch16] Gudrun von Schoenebeck. *Neue Schwerlast-Drohne Griff 300 fliegt mit 225 kg - ingenieur.de*. German. Dec. 2016. URL: <https://www.ingenieur.de/technik/fachbereiche/verkehr/neue-schwerlast-drohne-griff-300-fliegt-225-kg/> (visited on 10/27/2023) (see page 2).
- [Sha+19] Hazim Shakhathreh, Ahmad H. Sawalmeh, Ala Al-Fuqaha, Zuochao Dou, Eyad Almaita, Issa Khalil, Noor Shamsiah Othman, Abdallah Khreishah, and Mohsen Guizani. **Unmanned Aerial Vehicles (UAVs): A Survey on Civil Applications and Key Research Challenges**. *IEEE Access* 7 (2019), 48572–48634. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2019.2909530](https://doi.org/10.1109/ACCESS.2019.2909530). URL: <https://ieeexplore.ieee.org/document/8682048/> (see page 2).
- [ŞMK12] Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. **The Open Motion Planning Library**. *IEEE Robotics & Automation Magazine* 19:4 (Dec. 2012), 72–82. DOI: [10.1109/MRA.2012.2205651](https://doi.org/10.1109/MRA.2012.2205651) (see page 42).
- [Sun+22] Sihao Sun, Angel Romero, Philipp Foehn, Elia Kaufmann, and Davide Scaramuzza. *A Comparative Study of Nonlinear MPC and Differential-Flatness-Based Control for Quadrotor Agile Flight*. arXiv:2109.01365 [cs]. June 2022. URL: <http://arxiv.org/abs/2109.01365> (see pages 27, 29).
- [Sup+17] Bhakti Yudho Suprpto, M Ary Heryanto, Herwin Suprijono, Jemie Muiliadi, and Benyamin Kusumoputro. **Design and development of heavy-lift hexacopter for heavy payload**. In: *2017 International Seminar on Application for Technology of Information and Communication (iSemantic)*. Semarang: IEEE, Oct. 2017, 242–247. ISBN: 978-1-5386-3085-3. DOI: [10.1109/ISEMANTIC.2017.8251877](https://doi.org/10.1109/ISEMANTIC.2017.8251877). URL: <http://ieeexplore.ieee.org/document/8251877/> (see page 2).
- [Szo+22] Andrew Szot, Alex Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Chaplot, Oleksandr Maksymets, Aaron Gokaslan, Vladimir Vondrus, Sameer Dharur, Franziska

- Meier, Wojciech Galuba, Angel Chang, Zsolt Kira, Vladlen Koltun, Jitendra Malik, Manolis Savva, and Dhruv Batra. *Habitat 2.0: Training Home Assistants to Rearrange their Habitat*. arXiv:2106.14405 [cs]. July 2022. URL: <http://arxiv.org/abs/2106.14405> (see page 41).
- [Tou14] Marc Toussaint. *Newton methods for k-order Markov Constrained Motion Problems*. arXiv:1407.0414 [cs]. July 2014. URL: <http://arxiv.org/abs/1407.0414> (see page 11).
- [Yer+05] A. Yershova, L. Jaillet, T. Simeon, and S.M. LaValle. **Dynamic-Domain RRTs: Efficient Exploration by Controlling the Sampling Domain**. In: *Proceedings of the 2005 IEEE International Conference on Robotics and Automation*. Barcelona, Spain: IEEE, 2005, 3856–3861. ISBN: 978-0-7803-8914-4. DOI: 10.1109/ROBOT.2005.1570709. URL: <http://ieeexplore.ieee.org/document/1570709/> (see page 18).
- [Zha+22] Ketao Zhang, Pisak Chermprayong, Feng Xiao, Dimos Tzoumanikas, Barrie Dams, Sebastian Kay, Basaran Bahadir Kocer, Alec Burns, Lachlan Orr, Christopher Choi, Durgesh Dattatray Darekar, Wenbin Li, Steven Hirschmann, Valentina Soana, Shamsiah Awang Ngah, Sina Sareh, Ashutosh Choubey, Laura Margheri, Vijay M. Pawar, Richard J. Ball, Chris Williams, Paul Shepherd, Stefan Leutenegger, Robert Stuart-Smith, and Mirko Kovac. **Aerial additive manufacturing with multiple autonomous robots**. *Nature* 609:7928 (Sept. 2022). Number: 7928 Publisher: Nature Publishing Group, 709–717. ISSN: 1476-4687. DOI: 10.1038/s41586-022-04988-4. URL: <https://www.nature.com/articles/s41586-022-04988-4> (see page 2).

Declaration of Authorship

I hereby declare that this thesis is my own unaided work. All direct or indirect sources used are acknowledged as references.

Berlin, November 1, 2023

A handwritten signature in blue ink, consisting of a stylized 'T' followed by a dot and a 'K'.

Theo Valentin Kern